

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Hệ thống AIoT giám sát và dự báo
chất lượng không khí dựa trên
dữ liệu chuỗi thời gian**

Nguyễn Lê Hà Sơn

son.nlh225157@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Đỗ Công Thuần

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 6/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Hệ thống AIoT giám sát và dự báo
chất lượng không khí dựa trên
dữ liệu chuỗi thời gian**

Nguyễn Lê Hà Sơn

son.nlh225157@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Đỗ Công Thuận

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 6/2026

LỜI CAM KẾT

Họ và tên sinh viên: Nguyễn Lê Hà Sơn
Điện thoại liên lạc: 0867748664
Email: son.nlh225157@sis.hust.edu.vn
Lớp: Kỹ thuật máy tính 07 - K67
Hệ đào tạo: Cử nhân

Tôi - Nguyễn Lê Hà Sơn - cam kết Đồ án Tốt nghiệp (ĐATN) là công trình nghiên cứu của bản thân tôi dưới sự hướng dẫn của TS. Đỗ Công Thuận.

Các kết quả nêu trong ĐATN là trung thực, là thành quả của riêng tôi, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong ĐATN - bao gồm hình ảnh, bảng biểu, số liệu, và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Tôi xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày tháng năm

Tác giả ĐATN

Nguyễn Lê Hà Sơn

LỜI CẢM ƠN

Lời đầu tiên, em xin được gửi lời cảm ơn chân thành nhất đến giảng viên hướng dẫn là thầy Đỗ Công Thuận. Thầy đã dành rất nhiều thời gian, công sức để tận tình hướng dẫn em hoàn thành đồ án một cách tốt nhất. Qua các buổi trao đổi, em không chỉ được hướng dẫn về các kiến thức chuyên môn mà còn học hỏi được thêm rất nhiều điều như cách làm việc chuyên nghiệp, cách tiếp cận vấn đề cũng như tư duy phản biện.

Em cũng xin bày tỏ lòng biết ơn sâu sắc đến các giảng viên của Đại Học Bách Khoa Hà Nội, đặc biệt là các thầy cô của Trường Công nghệ Thông tin và Truyền thông. Những bài giảng, những tiết học của các thầy cô trong suốt 4 năm đại học đã giúp em có một nền tảng kiến thức vững chắc và ngày càng hoàn thiện hơn bản thân mình. Những giá trị này sẽ luôn là hành trang quý báu đối với em trong tất cả các chặng đường phía trước.

Đặc biệt, em cũng không thể không gửi lời cảm ơn đến gia đình, người thân và bạn bè đã luôn ủng hộ em vô điều kiện, luôn là chỗ dựa tinh thần vững chắc và cũng là niềm động lực to lớn để em cố gắng từng ngày. Những lời động viên, hỏi thăm tuy không quá đặc biệt nhưng vẫn luôn tiếp thêm sức mạnh cho em để vượt qua được những thời điểm khó khăn và có thể đi được đến ngày hôm nay.

Em xin cảm ơn những người bạn đã cùng đồng hành, động viên em trong suốt quá trình thực hiện đồ án tốt nghiệp. Những buổi gặp gỡ, nói chuyện đã giúp em thoải mái hơn rất nhiều về tinh thần khi đang trong giai đoạn căng thẳng của đồ án.

Cuối cùng, em xin cảm ơn chính bản thân mình đã luôn cố gắng, nỗ lực để hoàn thiện đồ án. Dù kiến thức có hạn chế, dù có nhiều vấn đề phát sinh thì bản thân em vẫn luôn tiến về phía trước và dù cho đồ án này vẫn còn tồn tại rất nhiều thiếu sót, thì em cũng đã hoàn thành đồ án tốt nghiệp với tất cả khả năng của mình.

Em xin chân thành cảm ơn!

Sinh viên thực hiện

TÓM TẮT NỘI DUNG ĐỒ ÁN

Ô nhiễm không khí đang là một trong những vấn đề môi trường nghiêm trọng nhất hiện nay, đặc biệt tại các đô thị lớn. Tuy nhiên, các hệ thống quan trắc chất lượng không khí hiện có thường có chi phí triển khai cao, mật độ trạm thưa và thiếu khả năng dự báo sớm. Đồ án trình bày việc thiết kế và triển khai một hệ thống IoT giám sát và dự báo chất lượng không khí theo thời gian thực, sử dụng các thiết bị phần cứng chi phí thấp kết hợp với mô hình học máy trên dữ liệu chuỗi thời gian.

Mạng lưới giám sát bao gồm các node cảm biến đo đạc các thông số nhiệt độ, độ ẩm và chất lượng không khí. Thiết bị hỗ trợ tự động cấu hình mạng không dây và đăng ký định danh tự động với máy chủ trung tâm khi khởi động, sau đó truyền tải dữ liệu đo đạc định kỳ về hệ thống lưu trữ trung tâm.

Hệ thống máy chủ trung tâm thực hiện việc tiếp nhận dữ liệu từ các thiết bị đo, xác thực thiết bị, lưu trữ dữ liệu chuỗi thời gian và tự động phát hiện các chỉ số vượt ngưỡng để gửi cảnh báo kịp thời cho người dùng. Đồng thời, dữ liệu đo đạc được phân phối tức thì đến giao diện giám sát trực quan của người dùng dưới dạng các biểu đồ thời gian thực.

Mô hình học sâu dự báo chuỗi thời gian được tích hợp để dự báo trước xu hướng thay đổi của các thông số môi trường trong tương lai gần dựa trên dữ liệu quá khứ. Kết hợp với các thuật toán nội suy địa thống kê, hệ thống có thể xây dựng bản đồ phân bố nhiệt độ và độ ẩm liên tục trong không gian phòng, hiển thị dưới dạng bản đồ nhiệt trực quan để xác định các vùng có điều kiện môi trường bất thường.

Kết quả triển khai cho thấy hệ thống có thể thu thập, lưu trữ, hiển thị thời gian thực, dự báo và trực quan hóa phân bố không gian trong một kiến trúc hoàn chỉnh, phù hợp cho nghiên cứu, giáo dục và mở rộng thành hệ thống giám sát môi trường thực tế.

MỤC LỤC

Lời cam kết	i
Lời cảm ơn	ii
Tóm tắt nội dung đồ án	iii
Danh mục thuật ngữ và từ viết tắt	x
1 GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu và phạm vi đề tài	2
1.2.1 Mục tiêu đề tài	2
1.2.2 Phạm vi đề tài	3
1.3 Đóng góp của đồ án	3
1.3.1 Xây dựng bản thiết kế tổng thể hệ thống AIoT quan trắc môi trường	3
1.3.2 Triển khai pipeline dự báo chuỗi thời gian tích hợp nội suy không gian	4
1.3.3 Triển khai thành công hệ thống prototype hoạt động đầy đủ	4
1.4 Bố cục đồ án	4
2 KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	6
2.1 Khảo sát hiện trạng	6
2.1.1 Tổng quan về nhu cầu quan trắc môi trường trong không gian kín . .	6
2.1.2 Các giải pháp hiện có trên thị trường	7
2.1.3 Xu hướng công nghệ AIoT trong quan trắc môi trường	8
2.1.4 Kết luận và đề xuất giải pháp	8
2.2 Tổng quan chức năng	8
2.2.1 Biểu đồ use case tổng quát	8
2.2.2 Biểu đồ use case phân rã: Đăng ký và cấu hình node IoT	9
2.2.3 Biểu đồ use case phân rã: Thu thập dữ liệu và dự báo	10

2.2.4	Biểu đồ use case phân rã: Xem dữ liệu và nhận cảnh báo	11
2.2.5	Quy trình nghiệp vụ	12
2.3	Đặc tả chức năng	16
2.3.1	Đặc tả use case UC01 - Đăng ký node IoT vào hệ thống	16
2.3.2	Đặc tả use case UC02 - Thu thập và lưu trữ dữ liệu cảm biến	17
2.3.3	Đặc tả use case UC03 - Xem dự báo chuỗi thời gian và heatmap không gian	18
2.4	Yêu cầu phi chức năng	18
3	THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG	20
3.1	Thiết kế kiến trúc	20
3.1.1	Lựa chọn kiến trúc phần mềm	20
3.1.2	Thiết kế tổng quan các gói	21
3.2	Thiết kế chi tiết	23
3.2.1	Thiết kế phần cứng node cảm biến	23
3.2.2	Thiết kế Firmware	25
3.2.3	Thiết kế cơ sở dữ liệu	29
3.2.4	Thiết kế Backend API	31
3.2.5	Thiết kế và huấn luyện mô hình Machine Learning	33
3.2.6	Thiết kế Dashboard (Frontend)	35
3.3	Triển khai	37
3.3.1	Môi trường triển khai	37
3.3.2	Hướng dẫn khởi động hệ thống	37
3.3.3	Script mô phỏng node IoT	38
3.4	Tổng hợp công nghệ sử dụng trong hệ thống	39
4	ĐÁNH GIÁ HỆ THỐNG VÀ CÁC GIẢI PHÁP NỔI BẬT	40
4.1	Kết quả và đánh giá hệ thống	40
4.1.1	Kết quả đạt được	40
4.1.2	Minh họa các chức năng chính	43

4.1.3	Kiểm thử hệ thống	44
4.2	Pipeline dự báo chuỗi thời gian LSTM qua FastAPI	46
4.2.1	Đặt vấn đề	46
4.2.2	Giải pháp	46
4.2.3	Đánh giá độ chính xác mô hình	47
4.3	Nội suy không gian Kriging/IDW để tạo heatmap phân bố	48
4.3.1	Đặt vấn đề	48
4.3.2	Giải pháp	49
4.4	Kiến trúc truyền thông MQTT kết hợp Socket.io đảm bảo độ trễ thấp	50
4.4.1	Đặt vấn đề	50
4.4.2	Giải pháp	50
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	53
5.1	Kết luận	53
5.1.1	Kết quả đạt được	53
5.1.2	Đánh giá so với mục tiêu	54
5.1.3	Hạn chế	54
5.2	Hướng phát triển	55
5.2.1	Mở rộng khả năng cảm biến	55
5.2.2	Nâng cao mô hình Machine Learning	55
5.2.3	Cải thiện nội suy không gian	55
5.2.4	Tối ưu bảo mật và hạ tầng	55
5.2.5	Tích hợp hệ sinh thái và mở rộng ứng dụng	55
5.2.6	Thử nghiệm và đánh giá quy mô lớn	56
	TÀI LIỆU THAM KHẢO	59
A	PHỤ LỤC: MÃ NGUỒN QUAN TRỌNG	60
A.1	Mã nguồn Firmware ESP32	60
A.2	Cấu trúc Cơ sở dữ liệu và Dịch vụ Cảnh báo Backend	61

A.3 Mã nguồn Python Machine Learning Service	64
--	----

DANH SÁCH HÌNH VẼ

2.1	Biểu đồ use case tổng quan của hệ thống	9
2.2	Biểu đồ use case phân rã: Đăng ký và cấu hình node IoT	10
2.3	Biểu đồ use case phân rã: Thu thập dữ liệu và dự báo	10
2.4	Biểu đồ use case phân rã: Xem dữ liệu và nhận cảnh báo	11
2.5	Quy trình nghiệp vụ thu thập và xử lý dữ liệu	13
2.6	Quy trình nghiệp vụ dự báo và nội suy không gian	15
3.1	Kiến trúc tổng quan của hệ thống AIoT giám sát và dự báo chất lượng không khí	21
3.2	Biểu đồ phụ thuộc gói tổng quan của hệ thống	22
3.3	Sơ đồ kết nối phần cứng node cảm biến	24
3.4	Kiến trúc firmware phân tầng của node ESP32	27
3.5	Sơ đồ ERD cơ sở dữ liệu PostgreSQL	30
3.6	Sơ đồ điều hướng các trang trong React dashboard	36
4.1	Giao diện chức năng hệ thống — Đăng nhập, đăng ký và quản lý thiết bị . .	41
4.2	Giao diện chức năng hệ thống — Dashboard và các tab chi tiết thiết bị . . .	42
4.3	Giao diện dự báo thời gian/không gian, quản lý phòng và cảnh báo hệ thống	43
4.4	Đường cong loss trong quá trình huấn luyện mô hình LSTM	48
4.5	Heatmap phân bố môi trường không gian được tạo từ nội suy Kriging, hiển thị trên bản đồ Leaflet	49
4.6	Luồng truyền thông end-to-end của hệ thống	51

DANH SÁCH BẢNG

2.1	So sánh các giải pháp quan trắc môi trường hiện có	7
2.2	Đặc tả use case UC01: Đăng ký node IoT vào hệ thống	16
2.3	Đặc tả use case UC02: Thu thập và lưu trữ dữ liệu cảm biến	17
2.4	Đặc tả use case UC03: Xem dự báo và heatmap không gian	18
3.1	Thông số kỹ thuật cảm biến DHT11	24
3.2	Cấu hình chân kết nối ESP32 — DHT11	25
3.3	Cấu trúc MQTT topic trong hệ thống	26
3.4	Cấu hình môi trường triển khai	37
3.5	Tổng hợp công nghệ sử dụng trong hệ thống	39
4.1	Kiểm thử chức năng đăng ký và kết nối node IoT	44
4.2	Kiểm thử chức năng thu thập và hiển thị dữ liệu realtime	45
4.3	Kiểm thử chức năng dự báo và heatmap không gian	45
4.4	Kiểm thử chức năng cảnh báo vượt ngưỡng	45
4.5	Sai số MAE của mô hình LSTM theo từng mốc dự báo	48

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence), mô phỏng các quá trình suy nghĩ và học tập của con người bằng máy móc
AIoT	Trí tuệ nhân tạo kết hợp Internet vạn vật (Artificial Intelligence of Things), tích hợp AI vào các thiết bị IoT để xử lý và phân tích dữ liệu tại chỗ
AQI	Chỉ số chất lượng không khí (Air Quality Index), thang đo mức độ ô nhiễm không khí ảnh hưởng đến sức khỏe con người
API	Giao diện lập trình ứng dụng (Application Programming Interface), tập hợp các giao thức cho phép các phần mềm giao tiếp với nhau
BLE	Công nghệ truyền thông không dây tầm ngắn tiêu thụ năng lượng thấp (Bluetooth Low Energy)
CNN	Mạng nơ-ron tích chập (Convolutional Neural Network), loại mạng nơ-ron thường dùng cho xử lý ảnh và tín hiệu tuần tự
CO ₂	Khí carbon dioxide, một trong các chất ô nhiễm không khí phổ biến
DHT22	Cảm biến nhiệt độ và độ ẩm kỹ thuật số độ chính xác cao
Edge Computing	Mô hình tính toán phân tán, xử lý dữ liệu gần nguồn phát sinh thay vì tập trung tại trung tâm dữ liệu
LSTM	Bộ nhớ ngắn hạn dài (Long Short-Term Memory), loại mạng nơ-ron hồi tiếp chuyên xử lý dữ liệu chuỗi thời gian
IoT	Internet vạn vật (Internet of Things), mạng lưới các thiết bị vật lý được kết nối internet để thu thập và chia sẻ dữ liệu
MAE	Sai số tuyệt đối trung bình (Mean Absolute Error), độ đo đánh giá hiệu năng mô hình dự báo
ML	Học máy (Machine Learning), lĩnh vực AI cho phép máy tính học từ dữ liệu mà không cần lập trình cụ thể
MQTT	Giao thức truyền tin nhẹ (Message Queuing Telemetry Transport), thường dùng trong các hệ thống IoT
MQ-135	Cảm biến khí đa năng, phát hiện NH ₃ , NO _x , cồn, benzen, khói và CO ₂
PM2.5	Hạt bụi mịn có đường kính nhỏ hơn hoặc bằng 2,5 micromet, gây hại nghiêm trọng cho đường hô hấp

Thuật ngữ	Ý nghĩa
PM10	Hạt bụi có đường kính nhỏ hơn hoặc bằng 10 micromet
PMS5003	Cảm biến bụi mịn laser đo PM1.0, PM2.5 và PM10
RMSE	Căn bậc hai của sai số bình phương trung bình (Root Mean Squared Error), độ đo đánh giá hiệu năng mô hình dự báo
TFLite	TensorFlow Lite, framework chạy mô hình ML trên thiết bị nhúng và di động với tài nguyên hạn chế
TinyML	Triển khai các mô hình học máy nhỏ gọn trực tiếp trên vi điều khiển có tài nguyên hạn chế

CHƯƠNG 1.

GIỚI THIỆU ĐỀ TÀI

Chương này trình bày tổng quan về đề án, làm rõ bối cảnh, động cơ, mục tiêu, phạm vi nghiên cứu và những đóng góp dự kiến. Xu hướng tích hợp trí tuệ nhân tạo vào hệ thống IoT (AIoT) và nhu cầu cấp thiết về giám sát, dự báo chất lượng không khí theo thời gian thực tạo ra cơ hội cho việc phát triển các giải pháp quan trắc môi trường thông minh với chi phí thấp. Đề án hướng đến việc xây dựng một hệ thống hoàn chỉnh gồm các node cảm biến IoT, pipeline xử lý dữ liệu chuỗi thời gian, mô hình dự báo LSTM và giao diện trực quan hóa phân bố không gian, chứng minh tính khả thi của AIoT trong lĩnh vực quan trắc môi trường đô thị.

1.1 Đặt vấn đề

Ô nhiễm không khí hiện là một trong những thách thức môi trường và y tế nghiêm trọng nhất của thế kỷ 21. Theo Tổ chức Y tế Thế giới (WHO), mỗi năm có khoảng 7 triệu người tử vong sớm do tiếp xúc với không khí ô nhiễm [1]. Tại Việt Nam, tình trạng ô nhiễm không khí tại các thành phố lớn như Hà Nội và Thành phố Hồ Chí Minh ngày càng trở nên nghiêm trọng, với chỉ số AQI thường xuyên vượt ngưỡng an toàn, đặc biệt trong các tháng mùa đông và những ngày không có gió [2].

Nhiệt độ và độ ẩm không khí là hai thông số môi trường cơ bản nhưng có mối tương quan chặt chẽ với chất lượng không khí và sức khỏe con người [1]. Độ ẩm cao kết hợp với nhiệt độ thích hợp tạo điều kiện cho sự phát triển của vi khuẩn, nấm mốc và tăng cường phản ứng hóa học của các chất ô nhiễm trong không khí. Việc giám sát liên tục, dự báo sớm xu hướng biến đổi của hai chỉ số này tại nhiều điểm phân tán trong không gian là nền tảng thiết yếu cho các hệ thống cảnh báo môi trường thông minh.

Tuy nhiên, các hệ thống quan trắc môi trường hiện có đang bộc lộ nhiều hạn chế đáng kể:

- **Chi phí cao và mật độ thấp:** Các trạm quan trắc tự động của nhà nước có giá lên đến hàng trăm triệu đồng mỗi trạm, dẫn đến mật độ trạm thưa thớt, không đủ để phản ánh sự phân bố môi trường theo không gian trong các tòa nhà hay khu vực quy mô nhỏ.
- **Thiếu khả năng dự báo:** Phần lớn các hệ thống chỉ dừng lại ở việc đo và hiển thị dữ liệu hiện tại, không có khả năng dự báo xu hướng trong tương lai để người dùng chủ động ứng phó.

- **Thiếu trực quan hóa phân bố không gian:** Các hệ thống thường hiển thị dữ liệu dạng số hoặc biểu đồ thời gian, chưa cung cấp bản đồ nhiệt (heatmap) thể hiện sự phân bố môi trường theo không gian thực, gây khó khăn cho việc xác định vùng cần can thiệp.
- **Hệ sinh thái đóng:** Nhiều sản phẩm thương mại không hỗ trợ tùy biến hoặc mở rộng, gây khó khăn cho nghiên cứu và tích hợp vào các hệ thống hiện có.

Sự phát triển của **Internet of Things (IoT)** cho phép triển khai mạng lưới node cảm biến phân tán chi phí thấp với khả năng kết nối và truyền dữ liệu liên tục. Kết hợp với các kỹ thuật **học máy** (Machine Learning), đặc biệt là các mô hình học sâu cho dữ liệu chuỗi thời gian như LSTM (Long Short-Term Memory), hệ thống có thể không chỉ quan trắc mà còn dự báo xu hướng môi trường trong tương lai. Hơn nữa, các thuật toán nội suy không gian như **Kriging** và **IDW** (Inverse Distance Weighting) cho phép tái tạo bản đồ phân bố môi trường liên tục từ một tập hợp hữu hạn các điểm đo, mở ra khả năng trực quan hóa trực quan và toàn diện hơn.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu đề tài

Đề án đặt ra mục tiêu xây dựng một **hệ thống AIoT hoàn chỉnh** có khả năng giám sát và dự báo chất lượng không khí (tập trung vào nhiệt độ, độ ẩm và AQI) trong một không gian xác định, bao gồm các mục tiêu cụ thể sau:

1. **Xây dựng node cảm biến IoT:** Thiết kế và triển khai các node cảm biến sử dụng vi điều khiển ESP32 tích hợp cảm biến DHT11 để thu thập dữ liệu nhiệt độ, độ ẩm và AQI mô phỏng định kỳ. Dữ liệu được truyền về hệ thống trung tâm qua giao thức MQTT qua kết nối Wi-Fi.
2. **Xây dựng hệ thống backend thu thập và lưu trữ:** Phát triển backend Node.js/Express tích hợp MQTT broker (Aedes) để tiếp nhận dữ liệu từ các node cảm biến, lưu trữ vào cơ sở dữ liệu PostgreSQL và phân phối theo thời gian thực đến các client thông qua Socket.io.
3. **Triển khai mô hình dự báo chuỗi thời gian:** Xây dựng và huấn luyện mô hình LSTM bằng PyTorch để dự báo nhiệt độ, độ ẩm và AQI trong 15 phút tới dựa trên 30 mẫu dữ liệu gần nhất. Mô hình được phục vụ thông qua một Python/FastAPI service độc lập.

4. **Nội suy và trực quan hóa phân bố không gian:** Từ dữ liệu thực đo và dữ liệu dự báo của các node, áp dụng thuật toán nội suy Kriging và IDW để tạo lưới dữ liệu liên tục (40×50 điểm), trực quan hóa dưới dạng heatmap trên bản đồ Leaflet.
5. **Xây dựng giao diện người dùng:** Phát triển dashboard web React để hiển thị dữ liệu thời gian thực qua biểu đồ Chart.js, kết quả dự báo, heatmap phân bố không gian và hệ thống cảnh báo khi các chỉ số vượt ngưỡng.
6. **Chi phí thấp và mã nguồn mở:** Sử dụng linh kiện phổ biến, giá rẻ và công bố mã nguồn mở để cộng đồng nghiên cứu có thể tái sử dụng và phát triển.

1.2.2 Phạm vi đề tài

Đề tài phát triển một sản phẩm prototype để chứng minh tính khả thi của các mục tiêu đề ra. Phạm vi nghiên cứu được giới hạn cụ thể như sau:

- **Phần cứng:** Sử dụng vi điều khiển ESP32 kết hợp cảm biến DHT11 để đo nhiệt độ, độ ẩm và sinh AQI mô phỏng. Đề tài không thiết kế PCB tùy chỉnh và không mở rộng sang các cảm biến khí khác (PM2.5, CO₂).
- **Mô hình ML:** Tập trung vào kiến trúc LSTM (PyTorch) cho bài toán dự báo đa biến (nhiệt độ, độ ẩm) với window size 30 và forecast horizon 15 phút. Mô hình chạy phía server (FastAPI), không triển khai trực tiếp trên vi điều khiển.
- **Nội suy không gian:** Áp dụng hai phương pháp IDW và Kriging (thư viện `pykrige`, `scipy`) cho không gian phòng/khu vực. Không thực hiện mô hình khí tượng hoặc CFD.
- **Dữ liệu:** Mô hình được huấn luyện trên dữ liệu thu thập từ hệ thống prototype trong môi trường thực nghiệm. Không sử dụng dữ liệu khí tượng quốc gia.
- **Triển khai:** Hệ thống được thử nghiệm trong môi trường phòng lab với một số node cảm biến. Không thực hiện triển khai quy mô lớn ngoài trời hay đánh giá độ chính xác theo tiêu chuẩn đo lường quốc gia.

1.3 Đóng góp của đề án

1.3.1 Xây dựng bản thiết kế tổng thể hệ thống AIoT quan trắc môi trường

Đề án đề xuất và xây dựng một bản thiết kế hoàn chỉnh cho hệ thống giám sát và dự báo môi trường dựa trên AIoT theo kiến trúc bốn tầng: tầng thu thập (node ESP32/DHT11),

tầng truyền dẫn (MQTT/Wi-Fi), tầng xử lý và lưu trữ (Node.js/Express, PostgreSQL, FastAPI/LSTM) và tầng hiển thị (React dashboard). Thiết kế được xây dựng theo hướng mô-đun hóa, sử dụng công nghệ phổ biến và chi phí thấp, cho phép tái sử dụng và mở rộng dễ dàng cho các ứng dụng quan trắc môi trường khác.

1.3.2 Triển khai pipeline dự báo chuỗi thời gian tích hợp nội suy không gian

Đồ án xây dựng thành công pipeline hoàn chỉnh từ huấn luyện mô hình LSTM (PyTorch) trên dữ liệu chuỗi thời gian thực, phục vụ inference qua FastAPI, đến kết hợp kết quả dự báo với thuật toán nội suy không gian Kriging/IDW để tạo ra heatmap phân bố môi trường dự báo. Đây là điểm kỹ thuật nổi bật, kết hợp dự báo thời gian và mô hình hóa không gian trong cùng một hệ thống thời gian thực.

1.3.3 Triển khai thành công hệ thống prototype hoạt động đầy đủ

Dựa trên bản thiết kế đề xuất, đồ án triển khai thành công một prototype hoạt động đầy đủ, bao gồm: các node cảm biến ESP32 giao tiếp qua MQTT, backend xử lý và phân phối dữ liệu thời gian thực, Python service chạy mô hình LSTM và tính toán heatmap, và dashboard web trực quan. Prototype vận hành ổn định trong điều kiện thử nghiệm, chứng minh tính khả thi của toàn bộ pipeline từ thu thập dữ liệu đến dự báo và trực quan hóa không gian.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau:

Chương 2 đi vào khảo sát yêu cầu thực tế của bài toán thông qua phân tích các hệ thống quan trắc môi trường hiện có, tích hợp các nghiên cứu liên quan và các nguồn tài liệu ngoài đầy đủ. Trên cơ sở khảo sát, các chức năng cốt lõi của hệ thống được xác định và biểu diễn qua biểu đồ use case kèm đặc tả chi tiết.

Chương 3 trình bày chi tiết quá trình thiết kế và triển khai hệ thống, bao gồm phân tích lựa chọn công nghệ đi kèm chi tiết về từng cấu phần triển khai phần cứng, thiết kế firmware, thiết kế cơ sở dữ liệu, thiết kế API backend, mô hình học sâu dự báo và trực quan hóa dashboard.

Chương 4 tập trung vào đánh giá hệ thống thông qua kết quả thực tế, các thử nghiệm kiểm thử chức năng và đánh giá chi tiết ba giải pháp kỹ thuật nổi bật: pipeline dự báo chuỗi thời gian LSTM phục vụ qua API độc lập, cơ chế nội suy không gian Kriging/IDW tạo bản đồ nhiệt thời gian thực, và hạ tầng truyền thông tối ưu hóa độ trễ qua MQTT kết hợp

WebSockets.

Chương 5 tổng kết quá trình thực hiện đề tài, đánh giá kết quả đạt được so với mục tiêu ban đầu và đề xuất các hướng phát triển trong tương lai nhằm nâng cấp và hoàn thiện hệ thống.

CHƯƠNG 2.

KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này đi sâu vào việc phân tích yêu cầu thực tế của bài toán thông qua việc khảo sát các hệ thống quan trắc môi trường hiện có trên thị trường. Trên cơ sở khảo sát, các chức năng cốt lõi cần có trong hệ thống sẽ được xác định rõ ràng và biểu diễn chi tiết thông qua biểu đồ use case cùng với phần đặc tả chức năng tương ứng.

2.1 Khảo sát hiện trạng

2.1.1 Tổng quan về nhu cầu quan trắc môi trường trong không gian kín

Chất lượng không khí trong các không gian kín như văn phòng, phòng học, nhà xưởng và tòa nhà thương mại đang nhận được sự quan tâm ngày càng tăng. Theo các nghiên cứu, con người dành trung bình hơn 90% thời gian trong nhà, trong khi chất lượng không khí trong nhà có thể kém hơn ngoài trời từ 2 đến 5 lần [1]. Sự phát triển của các mạng lưới cảm biến thông minh dựa trên IoT đã mở ra các hướng tiếp cận giám sát liên tục với độ phân giải không gian cao [3]. Hơn nữa, việc sử dụng các cảm biến chi phí thấp (LCS - Low-cost sensors) giúp phân tán mật độ trạm đo đặc tốt hơn, tuy nhiên lại đi kèm thách thức về sai số tích lũy và yêu cầu phải có quy trình hiệu chuẩn chặt chẽ để đảm bảo độ tin cậy của dữ liệu thu thập [4, 5]. Nhiệt độ và độ ẩm là hai thông số cơ bản nhưng có ảnh hưởng trực tiếp đến sức khỏe, năng suất làm việc và sự phát triển của vi sinh vật trong không gian.

Nhu cầu đặt ra cho một hệ thống quan trắc hiện đại không chỉ dừng ở việc **đo và hiển thị** giá trị hiện tại mà còn phải:

- Giám sát **liên tục, tự động** tại nhiều điểm phân tán trong không gian mà không cần can thiệp thủ công.
- **Dự báo xu hướng** trong tương lai gần để người vận hành chủ động điều chỉnh trước khi vượt ngưỡng nguy hiểm.
- Cung cấp **cái nhìn không gian** về sự phân bố nhiệt độ và độ ẩm trong toàn bộ khu vực, không chỉ tại từng điểm đơn lẻ.
- **Cảnh báo tự động** khi các chỉ số vượt ngưỡng an toàn đã cài đặt.
- Có thể **mở rộng** số lượng node cảm biến dễ dàng với chi phí thấp.

2.1.2 Các giải pháp hiện có trên thị trường

Hiện nay, thị trường cung cấp nhiều giải pháp quan trắc môi trường ở các mức độ khác nhau. Bảng 2.1 tổng hợp và so sánh các nhóm giải pháp tiêu biểu.

Bảng 2.1: So sánh các giải pháp quan trắc môi trường hiện có

Tiêu chí	Trạm quan trắc nhà nước	Thiết bị thương mại (Xiaomi, IQAir)	Giải pháp DIY / Nghiên cứu
Chi phí	Rất cao (vài tỷ–hàng chục tỷ VNĐ/trạm)	Trung bình (1,5 triệu–10 VNĐ/thiết bị)	Thấp (<500k VNĐ/node)
Mật độ triển khai	Thưa thớt	Vừa phải	Linh hoạt, tùy biến
Khả năng dự báo	Có (cloud-based)	Hạn chế hoặc không có	Tùy thiết kế
Phân bố không gian	Không	Không	Tùy thiết kế
Hệ sinh thái	Đóng	Đóng, phụ thuộc vendor	Mở, tùy biến
Phù hợp nghiên cứu	Không	Hạn chế	Cao

Các sản phẩm thương mại tiêu biểu như dòng máy đo của Xiaomi [6], thiết bị Qingping Air Monitor [7], hay thiết bị đo cao cấp IQAir AirVisual Pro [8] tích hợp các cảm biến chuyên dụng như SenseAir CO2 [9] tuy có thiết kế đẹp, hiển thị trực quan và dễ sử dụng cho người tiêu dùng cá nhân, nhưng hoạt động hoàn toàn trong hệ sinh thái đóng. Chúng không cho phép lập trình viên can thiệp vào firmware, tích hợp các pipeline học máy tùy biến để dự báo, hoặc liên kết dữ liệu giữa nhiều node để trực quan hóa bản đồ phân bố không gian theo thời gian thực. Đối với các trạm quan trắc tự động, liên tục của nhà nước, việc thiết lập và vận hành phải tuân thủ nghiêm ngặt theo các tiêu chuẩn kỹ thuật quy định bởi Bộ Tài nguyên và Môi trường [10]. Các trạm này tuy có độ chính xác và độ tin cậy rất cao nhờ các cảm biến chuẩn công nghiệp, nhưng chi phí đầu tư hạ tầng cực kỳ đắt đỏ, thường dao động từ vài tỷ đến hàng chục tỷ đồng cho mỗi trạm (ví dụ như dự án xây dựng mạng lưới quan trắc quốc gia giai đoạn 1 có tổng mức đầu tư lên tới 515 tỷ đồng cho 18 trạm cùng thiết bị phụ trợ [11]). Do chi phí lớn, mật độ lắp đặt các trạm này rất thưa thớt, chủ yếu đặt ở các nút giao thông hoặc khu công nghiệp lớn ngoài trời và hoàn toàn không thể sử dụng để giám sát chất lượng môi trường chi tiết bên trong các phòng học hay tòa nhà văn phòng. Từ đó, nhu cầu phát triển các giải pháp nghiên cứu tự thiết kế (DIY) dựa trên các nền tảng vi điều

khiến chi phí thấp kết hợp với thuật toán dự báo chuỗi thời gian và nội suy không gian đã trở thành xu hướng nghiên cứu quan trọng và thiết thực [12, 13].

2.1.3 Xu hướng công nghệ AIoT trong quan trắc môi trường

Sự phát triển của hệ sinh thái IoT với các vi điều khiển như ESP32 có tích hợp Wi-Fi và chi phí thấp đã mở ra hướng tiếp cận mới cho bài toán quan trắc phân tán [14]. Kết hợp với giao thức MQTT nhẹ và hiệu quả [15], các node cảm biến có thể truyền dữ liệu liên tục về hệ thống trung tâm với độ trễ thấp và tiêu thụ điện năng tối thiểu.

Về phía xử lý dữ liệu, các mô hình học sâu cho chuỗi thời gian - đặc biệt là LSTM (Long Short-Term Memory) [16] - đã chứng minh hiệu quả vượt trội so với các phương pháp thống kê truyền thống như ARIMA trong bài toán dự báo dữ liệu cảm biến môi trường. Kết hợp với các thuật toán nội suy không gian như Kriging [17] và IDW [18], hệ thống có thể tái tạo bản đồ phân bố môi trường liên tục từ một tập hợp hữu hạn các điểm đo.

2.1.4 Kết luận và đề xuất giải pháp

Từ phân tích trên, nhận thấy khoảng trống rõ ràng trên thị trường: chưa có giải pháp chi phí thấp, mã nguồn mở nào kết hợp được cả ba yếu tố: (1) giám sát thời gian thực đa node, (2) dự báo chuỗi thời gian bằng LSTM, và (3) trực quan hóa phân bố không gian bằng heatmap nội suy.

Đề tài đề xuất xây dựng hệ thống sử dụng vi điều khiển **ESP32** kết hợp cảm biến **DHT11**, giao tiếp qua **MQTT**, backend **Node.js/Express**, mô hình **LSTM** (PyTorch) phục vụ qua **FastAPI**, và nội suy không gian **Kriging/IDW** để tạo heatmap hiển thị trên **Leaflet**, nhằm lấp đầy khoảng trống đó.

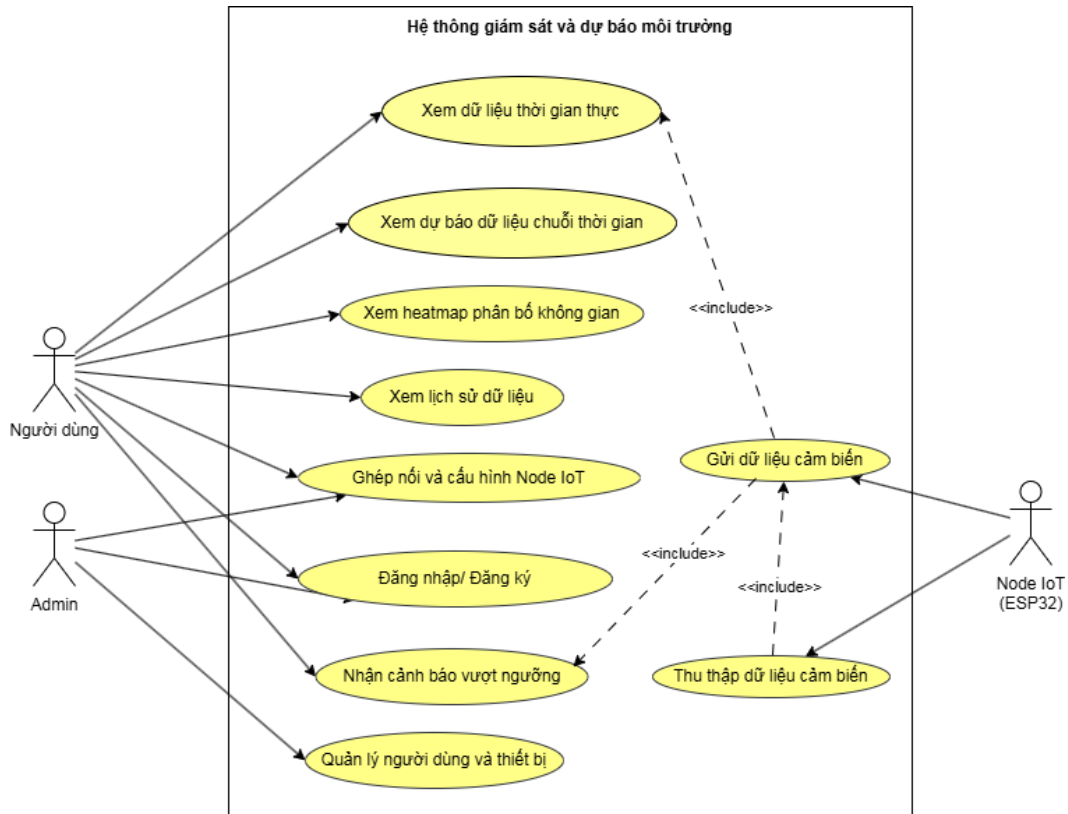
2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát

Hình 2.1 minh họa biểu đồ use case tổng quan của hệ thống, thể hiện mối quan hệ giữa các tác nhân và các chức năng chính. Hệ thống có hai tác nhân chính:

- **Người dùng (User):** Người vận hành hoặc quản lý không gian được giám sát. Tương tác chủ yếu qua giao diện web dashboard để xem dữ liệu, nhận cảnh báo và xem dự báo.

- **Node IoT (ESP32):** Thiết bị phần cứng đóng vai trò tác nhân tự động, định kỳ thu thập và gửi dữ liệu cảm biến lên hệ thống qua MQTT.

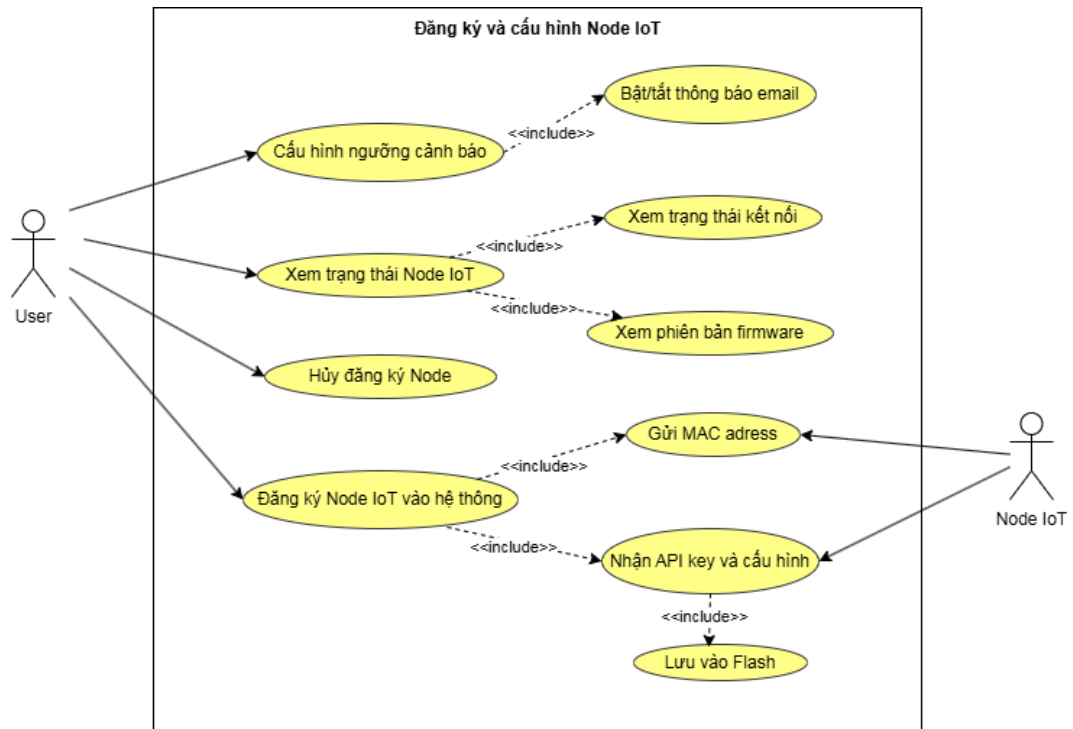


Hình 2.1: Biểu đồ use case tổng quan của hệ thống

Các chức năng chính của hệ thống bao gồm: đăng ký và cấu hình node IoT, thu thập dữ liệu cảm biến theo thời gian thực, xem dữ liệu và biểu đồ lịch sử, xem dự báo chuỗi thời gian, xem heatmap phân bố không gian, và nhận cảnh báo khi vượt ngưỡng.

2.2.2 Biểu đồ use case phân rã: Đăng ký và cấu hình node IoT

Hình 2.2 mô tả quy trình khởi tạo và quản lý node IoT trong hệ thống. Chức năng trọng tâm là quá trình *provisioning* tự động: node ESP32 khi khởi động lần đầu sẽ publish một thông điệp lên topic `devices/register` kèm địa chỉ MAC. Backend Aedes broker lắng nghe topic này, kiểm tra xem thiết bị đã tồn tại chưa, nếu chưa thì tạo bản ghi trong cơ sở dữ liệu, sinh `api_key` và gửi cấu hình (ngưỡng cảnh báo, phòng được gán) về node qua topic `devices/{mac_address}/config`.

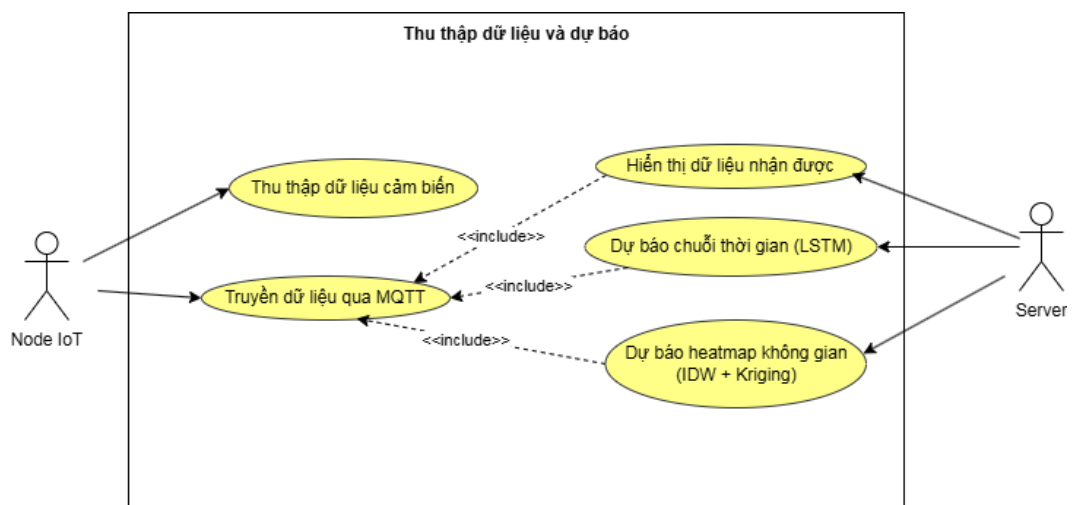


Hình 2.2: Biểu đồ use case phân rã: Đăng ký và cấu hình node IoT

Người dùng Admin có thể quản lý thiết bị qua dashboard: xem danh sách node, chỉnh sửa ngưỡng cảnh báo nhiệt độ, độ ẩm (cao/thấp), gán node vào phòng/khu vực, và vô hiệu hóa node khi cần.

2.2.3 Biểu đồ use case phân rã: Thu thập dữ liệu và dự báo

Hình 2.3 minh họa quy trình thu thập dữ liệu tự động từ node IoT và pipeline xử lý phía backend.

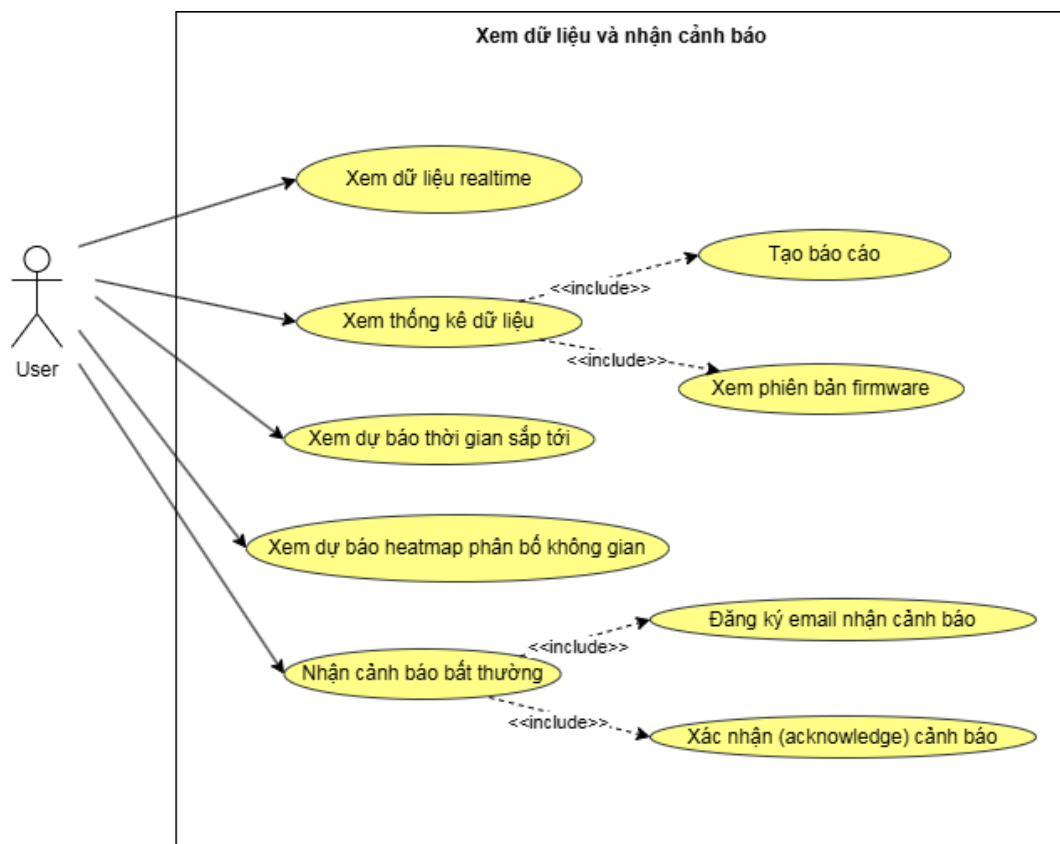


Hình 2.3: Biểu đồ use case phân rã: Thu thập dữ liệu và dự báo

Mỗi 5 giây, node ESP32 đọc dữ liệu từ cảm biến DHT11 và publish JSON lên topic `device/{api_key}/data`. Backend xác thực `api_key`, lưu vào bảng `readings` trong PostgreSQL, đồng thời emit sự kiện `new_reading` qua Socket.io đến các client đang subscribe. Khi người dùng yêu cầu xem dự báo, backend gọi FastAPI service, service này đọc 30 bản ghi gần nhất từ PostgreSQL, đưa qua mô hình LSTM và trả về 15 giá trị dự báo tiếp theo.

2.2.4 Biểu đồ use case phân rã: Xem dữ liệu và nhận cảnh báo

Hình 2.4 mô tả các chức năng tương tác chính của người dùng với dashboard.



Hình 2.4: Biểu đồ use case phân rã: Xem dữ liệu và nhận cảnh báo

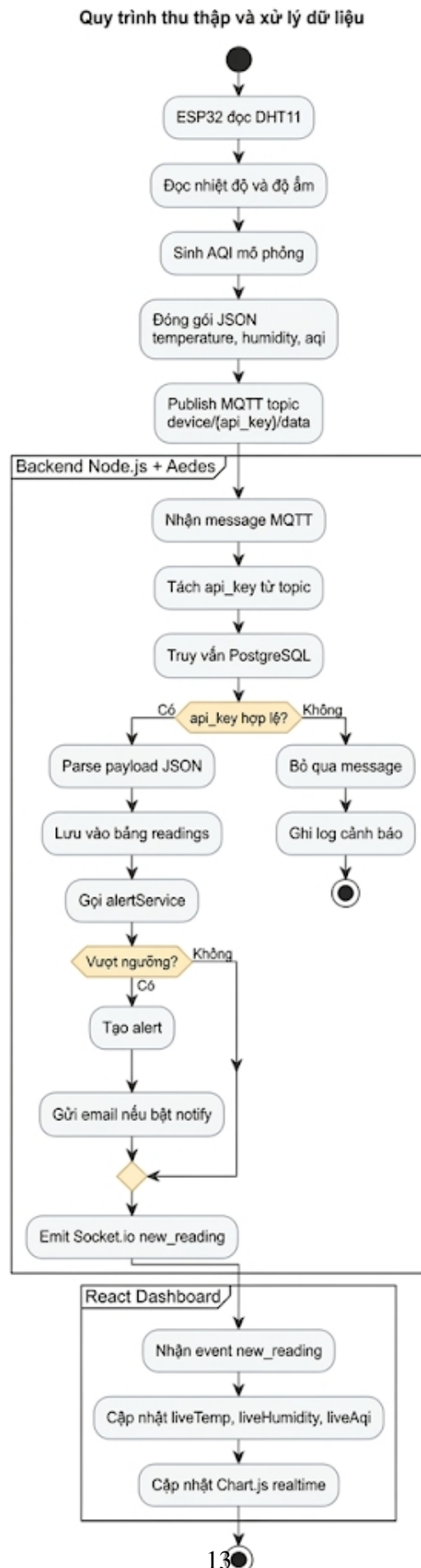
Người dùng có thể xem dữ liệu realtime qua biểu đồ Chart.js được cập nhật tức thì thông qua Socket.io, xem lịch sử dữ liệu theo khoảng thời gian tùy chọn, xem kết quả dự báo 15 phút tiếp theo, và xem heatmap phân bố nhiệt độ, độ ẩm trong không gian phòng trên bản đồ Leaflet. Khi các chỉ số vượt ngưỡng đã cài đặt, hệ thống tự động ghi alert vào cơ sở dữ liệu, hiển thị thông báo trên dashboard và gửi email cảnh báo.

2.2.5 Quy trình nghiệp vụ

a. Quy trình thu thập và xử lý dữ liệu

Hình 2.5 mô tả quy trình nghiệp vụ chính khi node IoT gửi dữ liệu về hệ thống. Quy trình diễn ra hoàn toàn tự động và liên tục theo chu kỳ 5 giây:

1. Node ESP32 đọc nhiệt độ, độ ẩm từ DHT11 và sinh giá trị AQI mô phỏng.
2. ESP32 đóng gói JSON gồm nhiệt độ, độ ẩm, AQI và publish lên MQTT broker (Aedes) qua topic `device/{api_key}/data`.
3. Backend lắng nghe sự kiện `publish` của Aedes, xác thực `api_key` với PostgreSQL.
4. Nếu hợp lệ, bản ghi được lưu vào bảng `readings`.
5. `alertService` kiểm tra giá trị vừa nhận với ngưỡng cài đặt của thiết bị. Nếu vượt ngưỡng, tạo bản ghi trong bảng `alerts` và gửi email thông báo.
6. Socket.io emit sự kiện `new_reading` đến tất cả client đang lắng nghe room `device_{id}` và `user_{id}`.
7. Dashboard React nhận sự kiện, cập nhật biểu đồ Chart.js mà không cần tải lại trang.

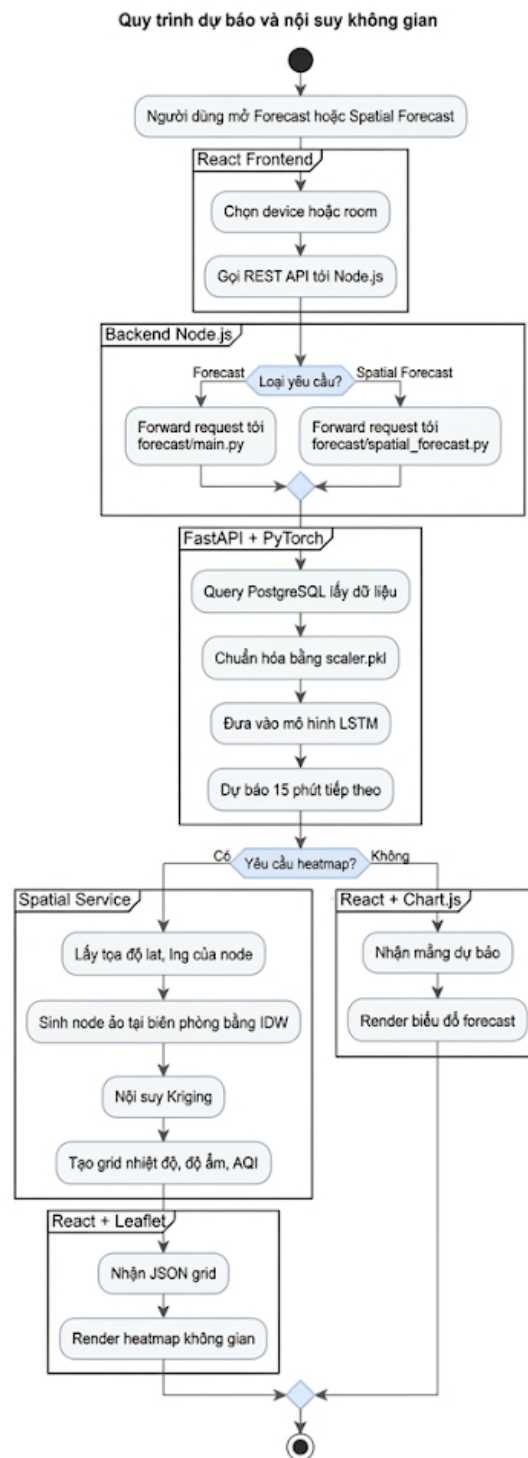


Hình 2.5: Quy trình nghiệp vụ thu thập và xử lý dữ liệu

b. Quy trình dự báo và nội suy không gian

Hình 2.6 mô tả quy trình khi người dùng yêu cầu xem dự báo và heatmap phân bố không gian:

1. Người dùng mở trang dự báo hoặc trang heatmap trên dashboard.
2. Frontend gọi REST API `GET /api/forecast` hoặc `GET /api/spatial-forecast` đến backend Node.js.
3. Backend chuyển tiếp request tới FastAPI service: `forecast/main.py` cho dự báo trạm hoặc `forecast/spatial_forecast.py` cho heatmap.
4. FastAPI service truy vấn dữ liệu lịch sử 30 phút gần nhất từ PostgreSQL.
5. Dữ liệu được chuẩn hóa qua `scaler.pkl`, đưa vào mô hình LSTM (PyTorch) để dự báo 15 phút tiếp theo.
6. Với yêu cầu heatmap: kết hợp giá trị thực đo (hoặc dự báo) với tọa độ vị trí của từng node, áp dụng thuật toán IDW và Kriging để nội suy ra lưới dữ liệu 40×50 điểm.
7. Kết quả trả về dưới dạng JSON, frontend render lên bản đồ Leaflet dưới dạng heatmap màu sắc.



Hình 2.6: Quy trình nghiệp vụ dự báo và nội suy không gian

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case UC01 - Đăng ký node IoT vào hệ thống

Bảng 2.2: Đặc tả use case UC01: Đăng ký node IoT vào hệ thống

Mã use case	UC01
Tên	Đăng ký node IoT vào hệ thống
Tác nhân	Node IoT (ESP32)
Mục đích	Cấp phát định danh và cấu hình ban đầu cho node cảm biến mới
Tiền điều kiện	- ESP32 đã được cấp thông tin Wi-Fi qua AP cấu hình <code>Sensor-Setup-[MAC]</code> - MQTT broker (Aedes) đang chạy và cho phép client provisioning kết nối
Luồng chính	- ESP32 khởi động, publish MAC address lên topic <code>devices/register</code> - Aedes broker nhận message, kiểm tra MAC trong DB - Nếu chưa tồn tại: tạo bản ghi device, sinh <code>api_key</code> ngẫu nhiên, lưu vào bảng <code>devices</code> - Backend publish cấu hình (ngưỡng, <code>api_key</code>) về topic <code>devices/{mac}/config</code> - ESP32 nhận cấu hình, lưu vào Flash qua <code>Preferences.h</code>
Luồng thay thế	MAC đã tồn tại: trả về cấu hình hiện có mà không tạo bản ghi mới
Hậu điều kiện	- Wi-Fi không kết nối được: ESP32 retry sau 5 giây - Node được đăng ký trong hệ thống với <code>api_key</code> duy nhất - Node sẵn sàng gửi dữ liệu lên topic <code>device/{api_key}/data</code>

2.3.2 Đặc tả use case UC02 - Thu thập và lưu trữ dữ liệu cảm biến

Bảng 2.3: Đặc tả use case UC02: Thu thập và lưu trữ dữ liệu cảm biến

Mã use case	UC02
Tên	Thu thập và lưu trữ dữ liệu cảm biến
Tác nhân	Node IoT (ESP32), Hệ thống tự động
Mục đích	Đọc dữ liệu môi trường định kỳ và lưu trữ vào cơ sở dữ liệu
Tiền điều kiện	1. Node đã đăng ký thành công (UC01) 2. Kết nối MQTT giữa node và broker ổn định
Luồng chính	1. Mỗi 5 giây, ESP32 đọc nhiệt độ, độ ẩm từ DHT11 và sinh AQI mô phỏng 2. Đóng gói thành JSON: <code>{"temperature": x, "humidity": y, "aqi": z}</code> 3. Publish lên topic <code>device/{api_key}/data</code> 4. Aedes broker xác thực <code>api_key</code>, lấy <code>device_id</code> từ PostgreSQL 5. Lưu bản ghi vào bảng <code>readings</code> với timestamp 6. Gọi <code>alertService</code> kiểm tra ngưỡng 7. Emit <code>new_reading</code> qua Socket.io
Luồng thay thế	4a. <code>api_key</code> không hợp lệ: bỏ qua message, ghi log cảnh báo
Hậu điều kiện	1a. DHT11 lỗi đọc: bỏ qua chu kỳ này, thử lại sau 5 giây 1. Dữ liệu được lưu bền vững trong PostgreSQL 2. Các client kết nối nhận được dữ liệu realtime qua Socket.io

2.3.3 Đặc tả use case UC03 - Xem dự báo chuỗi thời gian và heatmap không gian

Bảng 2.4: Đặc tả use case UC03: Xem dự báo và heatmap không gian

Mã use case	UC03
Tên	Xem dự báo chuỗi thời gian và heatmap không gian
Tác nhân	Người dùng (User)
Mục đích	Cung cấp thông tin dự báo xu hướng môi trường và phân bố không gian để hỗ trợ quyết định
Tiền điều kiện	1. Người dùng đã đăng nhập 2. Có ít nhất 30 bản ghi trong PostgreSQL cho node cần dự báo 3. FastAPI service đang chạy
Luồng chính	1. Người dùng chọn node và nhấn “Xem dự báo” 2. Frontend gọi GET <code>/api/forecast?device_id={id}</code> 3. Backend Node.js gọi FastAPI service (cổng 8000) 4. FastAPI đọc 30 bản ghi gần nhất, chuẩn hóa qua <code>scaler.pkl</code> 5. Mô hình LSTM suy luận, trả về mảng 15 giá trị dự báo 6. Frontend hiển thị biểu đồ dự báo kết hợp với dữ liệu lịch sử 7. Người dùng chọn tab “Heatmap”: frontend gọi GET <code>/api/spatial-forecast</code> 8. FastAPI tính Kriging/IDW, trả về lưới 40×50 điểm 9. Leaflet render heatmap màu sắc trên bản đồ phòng
Hậu điều kiện	Người dùng xem được dự báo 15 phút tới và bản đồ nhiệt phân bố không gian

2.4 Yêu cầu phi chức năng

Hệ thống được xây dựng cần đảm bảo các yêu cầu phi chức năng sau:

Hiệu năng: Độ trễ từ khi node gửi dữ liệu đến khi dashboard cập nhật không vượt quá 2 giây trong điều kiện mạng bình thường. Thời gian phản hồi của các API thông thường

dưới 500ms. Thời gian suy luận LSTM (15 phút dự báo) dưới 1 giây. Hệ thống hỗ trợ tối thiểu 10 node IoT hoạt động đồng thời.

Độ tin cậy: Dữ liệu cảm biến phải được lưu trữ đầy đủ, không mất mát trong điều kiện mạng ổn định. Khi node IoT mất kết nối MQTT, thiết bị tự động thử kết nối lại (reconnect) theo cơ chế backoff. Hệ thống cảnh báo phải gửi email trong vòng 30 giây kể từ khi phát hiện vượt ngưỡng.

Bảo mật: Tất cả API endpoint yêu cầu xác thực JWT. Mật khẩu người dùng được mã hóa bằng bcrypt trước khi lưu vào cơ sở dữ liệu. Mỗi node IoT được định danh bằng `api_key` duy nhất, được xác thực tại backend trước khi chấp nhận dữ liệu.

Khả năng sử dụng: Giao diện dashboard phải trực quan, hiển thị dữ liệu realtime mà không cần tải lại trang. Biểu đồ và heatmap phải phản hồi ngay lập tức khi có dữ liệu mới. Hệ thống cảnh báo hiển thị rõ ràng thông tin node, loại cảnh báo và thời điểm xảy ra.

Khả năng mở rộng: Kiến trúc phải cho phép bổ sung node cảm biến mới mà không cần thay đổi code backend. Cơ sở dữ liệu PostgreSQL được thiết kế để hỗ trợ truy vấn dữ liệu lịch sử hiệu quả với chỉ mục phù hợp trên cột `timestamp` và `device_id`.

Chi phí: Tổng chi phí phần cứng cho một node cảm biến (ESP32 + DHT11 + nguồn) không vượt quá 150.000 VND, đảm bảo khả năng triển khai mật độ cao với ngân sách hạn chế.

CHƯƠNG 3.

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

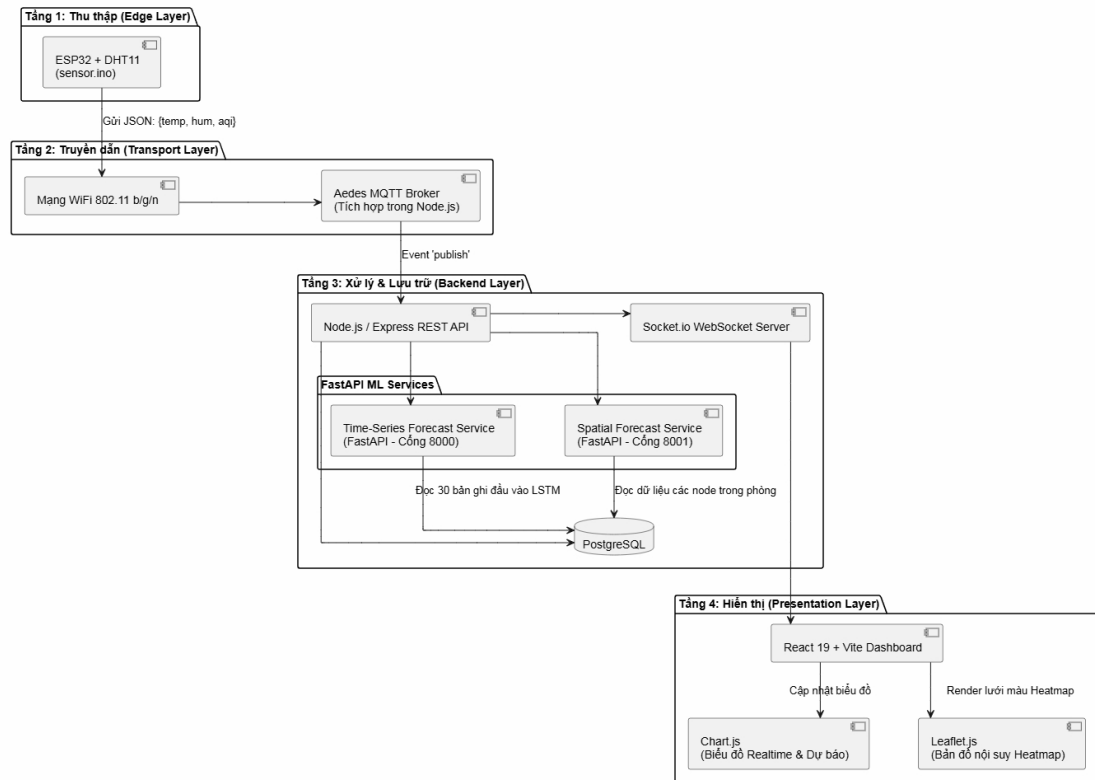
Chương này trình bày toàn bộ quá trình thiết kế và xây dựng hệ thống AIoT giám sát và dự báo chất lượng không khí. Nội dung bao gồm thiết kế kiến trúc tổng thể theo mô hình bốn tầng, thiết kế chi tiết từng thành phần tích hợp chặt chẽ với các phân tích lựa chọn công nghệ tương ứng (vi điều khiển, cảm biến, giao thức truyền thông, cơ sở dữ liệu, backend, mô hình dự báo học sâu và giao diện trực quan), kết quả xây dựng hệ thống và quy trình triển khai chi tiết.

3.1 Thiết kế kiến trúc

3.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế theo **kiến trúc bốn tầng** kết hợp mô hình IoT Gateway, thể hiện trong Hình 3.1. Kiến trúc này phân tách rõ ràng trách nhiệm của từng tầng, đảm bảo khả năng mở rộng độc lập và dễ bảo trì:

1. **Tầng thu thập (Edge Layer):** Node cảm biến đọc dữ liệu và gửi lên qua kết nối mạng không dây. Tầng này hoạt động hoàn toàn tự động, không cần can thiệp người dùng sau khi cấu hình ban đầu.
2. **Tầng truyền dẫn (Transport Layer):** Giao thức truyền thông nhẹ đảm bảo truyền dữ liệu nhanh, độ trễ thấp và hỗ trợ nhiều node đồng thời.
3. **Tầng xử lý và lưu trữ (Backend Layer):** Hệ thống máy chủ trung tâm xử lý nghiệp vụ, cơ sở dữ liệu lưu trữ dữ liệu chuỗi thời gian, và các API microservice chạy mô hình học sâu suy luận và thuật toán nội suy không gian.
4. **Tầng hiển thị (Presentation Layer):** Giao diện Web Dashboard nhận dữ liệu và cập nhật trực quan realtime dưới dạng biểu đồ và bản đồ nhiệt.



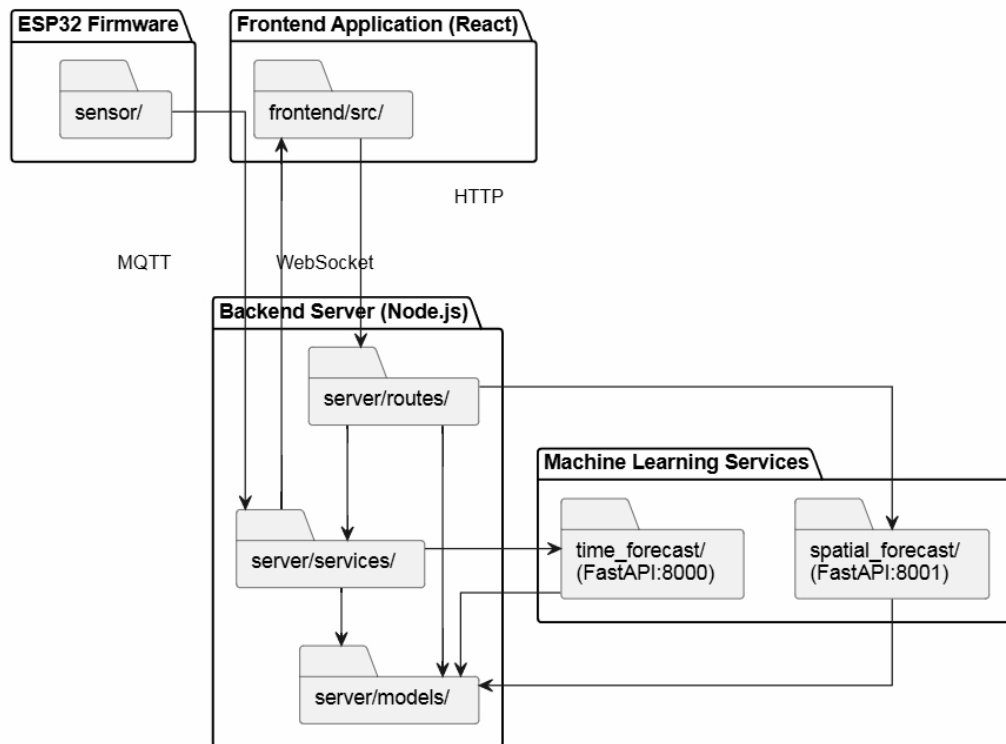
Hình 3.1: Kiến trúc tổng quan của hệ thống AIoT giám sát và dự báo chất lượng không khí

Lý do lựa chọn kiến trúc này so với các phương án khác:

- **So với kiến trúc monolithic:** Việc tách riêng dịch vụ Machine Learning độc lập khỏi máy chủ backend chính giúp đảm bảo hiệu năng xử lý, tránh block luồng chạy chính khi chạy suy luận nặng.
- **So với kiến trúc serverless:** Hệ thống cần duy trì kết nối truyền thông liên tục và tức thời (realtime), không phù hợp với mô hình stateless của serverless.
- **So với Cloud-only:** Broker tích hợp trực tiếp trong backend giúp giảm bớt một hop mạng trung gian, từ đó giảm thiểu độ trễ truyền dữ liệu đầu cuối.

3.1.2 Thiết kế tổng quan các gói

Hình 3.2 mô tả biểu đồ phụ thuộc gói tổng quan của hệ thống, phân chia thành bốn nhóm chính tương ứng với bốn tầng kiến trúc.



Hình 3.2: Biểu đồ phụ thuộc gói tổng quan của hệ thống

Cấu trúc thư mục dự án phản ánh trực tiếp kiến trúc này:

- `sensor/` — Firmware thiết bị vi điều khiển (C/C++)
- `server/` — Backend chính điều phối nghiệp vụ (JavaScript)
 - `controllers/` — Xử lý HTTP request
 - `routes/` — Định nghĩa API endpoint
 - `services/` — Logic nghiệp vụ (cảnh báo, kết nối truyền thông, dự báo)
 - `middleware/` — Xác thực, phân quyền bảo mật
 - `models/` — Giao tiếp cơ sở dữ liệu
- `lstm_train/` — Pipeline Machine Learning (huấn luyện, suy luận và nội suy)
- `frontend/src/` — Mã nguồn giao diện Web Dashboard (React)
 - `pages/` — Các màn hình chức năng chính
 - `components/` — Các thành phần UI dùng lại
 - `context/` — Quản lý trạng thái toàn cục

3.2 Thiết kế chi tiết

3.2.1 Thiết kế phần cứng node cảm biến

a. Lựa chọn và đặc tả chi tiết linh kiện phần cứng

Node cảm biến được thiết kế dựa trên sự kết hợp giữa hiệu năng xử lý, khả năng kết nối mạng không dây và tối ưu hóa chi phí triển khai thực tế.

- **Vi điều khiển ESP32:** ESP32 là dòng vi điều khiển (SoC) do Espressif Systems phát triển, tích hợp bộ xử lý lõi kép Xtensa LX6 32-bit hoạt động ở tần số lên đến 240 MHz, cùng với Wi-Fi 802.11 b/g/n và Bluetooth 4.2/BLE trên cùng một chip [19]. Các thông số kỹ thuật quan trọng bao gồm 520 KB SRAM, hỗ trợ Flash ngoài thông thường 4–16 MB, và đầy đủ các giao tiếp ngoại vi như I2C, SPI, UART, ADC, DAC.

Lý do lựa chọn cho đồ án:

- **Tích hợp Wi-Fi sẵn:** Cho phép kết nối MQTT trực tiếp đến broker mà không cần module truyền thông bổ sung, giảm chi phí và độ phức tạp phần cứng.
 - **Chi phí thấp:** Giá thành module ESP32 trên thị trường Việt Nam khoảng 50.000–80.000 VND, phù hợp triển khai nhiều node.
 - **Hệ sinh thái phong phú:** Được hỗ trợ đầy đủ bởi Arduino IDE với thư viện MQTT (PubSubClient), JSON (ArduinoJson) và cảm biến DHT đã trưởng thành.
 - **Bộ nhớ Flash đủ dùng:** Thư viện Preferences.h cho phép lưu cấu hình định danh (device_id, api_key và địa chỉ mqtt_server) vào Flash bền vững qua các lần reset.
- **Cảm biến DHT11:** DHT11 là cảm biến kỹ thuật số đo nhiệt độ và độ ẩm tương đối sử dụng giao tiếp 1-Wire [20]. Thông số kỹ thuật chính được thể hiện ở Bảng 3.1.

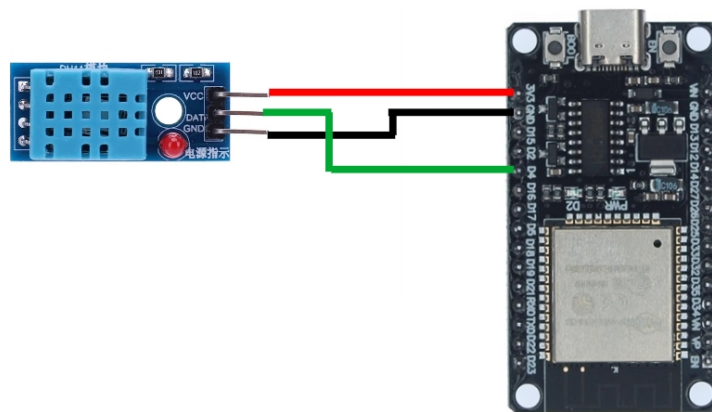
Bảng 3.1: Thông số kỹ thuật cảm biến DHT11

Thông số	Giá trị	Đơn vị
Dải đo nhiệt độ	0 - 50	°C
Độ chính xác nhiệt độ	± 2	°C
Dải đo độ ẩm	20 - 80	%RH
Độ chính xác độ ẩm	± 5	%RH
Điện áp hoạt động	3.3 - 5	V
Giao tiếp	1-Wire (Digital)	-
Tần suất lấy mẫu	1	lần/giây tối đa

DHT11 được lựa chọn vì giá thành rất thấp (khoảng 15.000–25.000 VND), dễ kết nối với ESP32 qua một chân digital duy nhất (GPIO4 trong đồ án), và thư viện `DHT.h` đã ổn định, dễ tích hợp. Mặc dù độ chính xác không bằng DHT22, DHT11 đủ đáp ứng yêu cầu của hệ thống quan trắc môi trường prototype trong phạm vi đồ án.

b. Sơ đồ kết nối phần cứng

Node cảm biến kết nối cảm biến DHT11 trực tiếp tới một chân đầu vào kỹ thuật số (GPIO4) của ESP32 qua điện trở kéo lên (pull-up) 10k Ω để đảm bảo tính ổn định của đường truyền 1-Wire. Sơ đồ kết nối chi tiết được minh họa trong Hình 3.3.

**Hình 3.3:** Sơ đồ kết nối phần cứng node cảm biến

Bảng 3.2 liệt kê cấu hình chân kết nối:

Bảng 3.2: Cấu hình chân kết nối ESP32 — DHT11

Chân DHT11	Chân ESP32	Chức năng	Ghi chú
VCC	3.3V	Nguồn cấp	—
DATA	GPIO4	Dữ liệu 1-Wire	Qua điện trở 10kΩ pull-up
GND	GND	Nối đất	—

c. Cơ chế lưu cấu hình định danh trên Flash

Để node cảm biến hoạt động tự động sau khi khởi động lại hoặc mất nguồn điện, thiết bị cần lưu trữ mã khóa xác thực định danh (`api_key`) và cấu hình nhận từ máy chủ. Chúng tôi sử dụng thư viện `Preferences.h` của ESP32 để ghi dữ liệu trực tiếp vào phân vùng NVS (Non-Volatile Storage) trên bộ nhớ Flash. Thuật toán đọc và ghi cấu hình được mô tả dưới dạng mã giả ở mã nguồn giải thuật bên dưới:

```

1 Ham docCauHinhTuFlash()
2     Mo vung luu tru "device" o che do chi doc (read-only)
3     deviceId = Doc gia tri so nguyen voi khoa "device_id" (mac dinh
4         = -1)
5     deviceApiKey = Doc chuoi ky tu voi khoa "api_key" (mac dinh = "
6         ")
7     Dong vung luu tru
8 Ket thuc Ham
9
10 Ham ghiCauHinhVaoFlash(idMoi, keyMoi)
11     Mo vung luu tru "device" o che do ghi-doc (read-write)
12     Ghi idMoi vao khoa "device_id"
13     Ghi keyMoi vao khoa "api_key"
14     Dong vung luu tru
15 Ket thuc Ham

```

Listing 3.1: Giai thuat luu va doc cau hinh dinh danh tren Flash

3.2.2 Thiết kế Firmware

a. Giao thức truyền thông MQTT và cấu trúc Topics

Hệ thống sử dụng giao thức truyền thông **MQTT** (Message Queuing Telemetry Transport) cho việc truyền dẫn dữ liệu từ các node cảm biến lên máy chủ [15]. MQTT hoạt động trên mô hình Publish/Subscribe qua nền tảng TCP/IP, có kích thước tiêu đề gói tin

(overhead) cực nhẹ (chỉ tối thiểu 2 byte), giúp tối ưu hóa băng thông truyền tải và giảm điện năng tiêu thụ cho thiết bị. Hệ thống sử dụng thư viện `PubSubClient` tích hợp trên ESP32 để kết nối đến máy chủ trung tâm. Cấu trúc các topic truyền tin được định nghĩa trong Bảng 3.3.

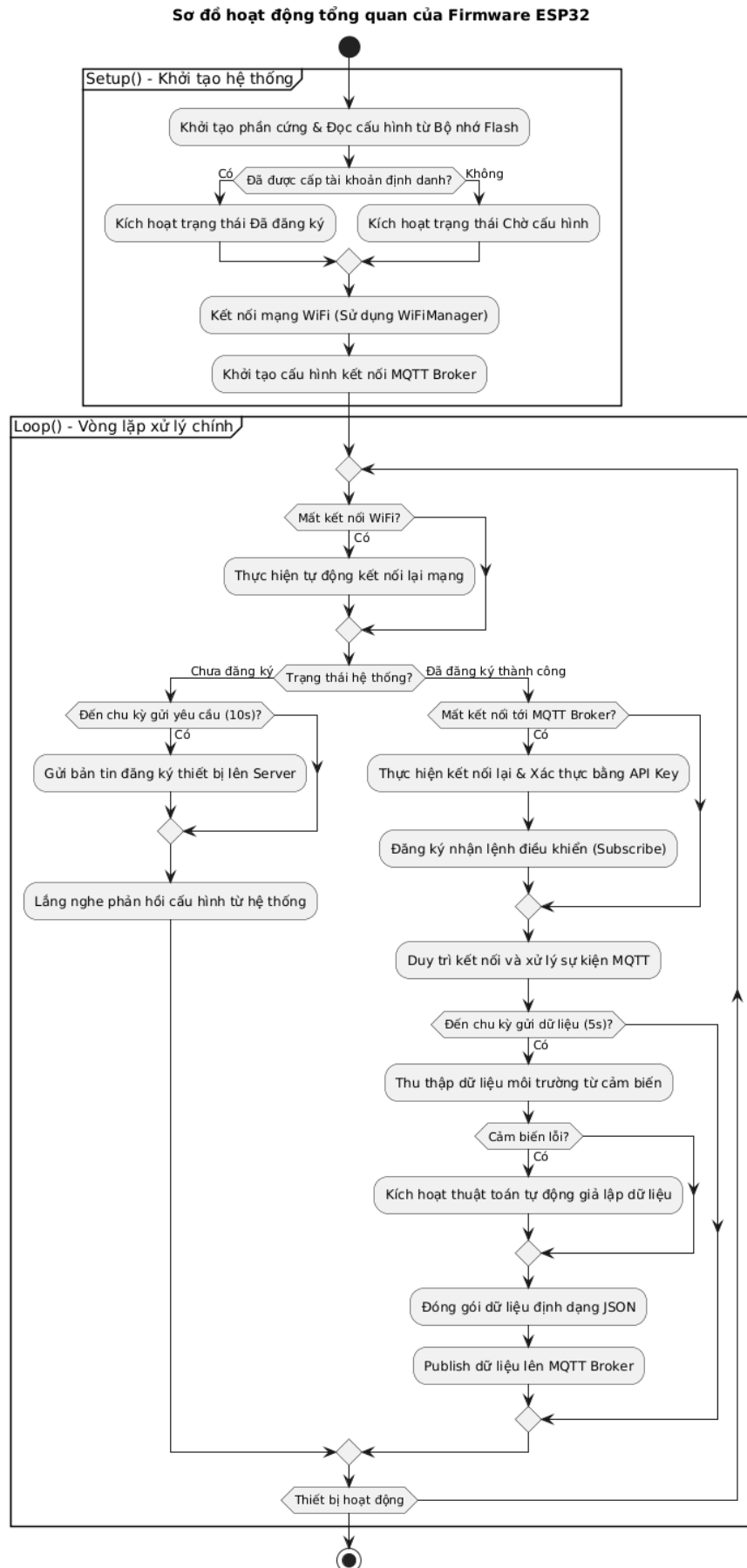
Bảng 3.3: Cấu trúc MQTT topic trong hệ thống

Topic	Chiều	Mô tả
<code>devices/register</code>	Node → Máy chủ	Node mới gửi MAC address để đăng ký tự động
<code>devices/{mac}/config</code>	Máy chủ → Node	Máy chủ trả lời <code>api_key</code> và thông số cấu hình
<code>device/{api_key}/data</code>	Node → Máy chủ	Gửi dữ liệu nhiệt độ, độ ẩm và AQI định kỳ

MQTT broker được tích hợp trực tiếp vào backend Node.js thông qua thư viện **Aedes** - một broker MQTT thuần Node.js hiệu năng cao, không yêu cầu cài đặt broker riêng biệt (như Mosquitto), giúp đơn giản hóa việc triển khai và tích hợp chặt chẽ với logic nghiệp vụ.

b. Luồng xử lý chính của firmware

Firmware được tổ chức theo kiến trúc phân tầng như minh họa trong Hình 3.4. Vòng lặp chính (`loop()`) điều phối ba tác vụ theo chu kỳ: đọc cảm biến, duy trì kết nối MQTT và gửi dữ liệu.



Hình 3.4: Kiến trúc firmware phân tầng của node ESP32

Giải thuật vận hành chính của firmware được mô tả qua mã giả dưới đây:

```

1 Ham setup()
2     Khởi tạo giao tiếp cảm biến DHT
3     docCauHinhTuFlash()
4     Neu chua co thong tin Wi-Fi:
5         Khởi chạy AP "Sensor-Setup-[MAC]" để người dùng cấu hình
           mạng
6     Cấu hình kết nối tới máy chủ MQTT Broker và đăng ký hàm
           callback nhận tin
7 Ket thuc Ham
8
9 Ham loop()
10    Neu chua co thong tin dang ky (api_key trong):
11        guiYeuCauDangKyToiTopic("devices/register") kèm địa chỉ MAC
12        Cho 5 giây rồi thu lại
13        Tra ve
14    Neu ket noi MQTT bi ngat:
15        Thuc hien ket noi lai bang tai khoan xac thuc api_key
16    Duy tri trang thai ket noi MQTT (mqttClient.loop())
17
18    Neu thoi gian hien tai - thoi diem gui cuoi cung >= 5 giây:
19        nhietDo = docNhietDoTuCamBien()
20        doAm = docDoAmTuCamBien()
21        aqi = sinhDuLieuAQIMoPhong() // Danh cho pipeline ML
22
23    Neu nhietDo va doAm hop le (khong phai NaN):
24        goiTin = Dong goi JSON { "temperature": nhietDo, "
           humidity": doAm, "aqi": aqi }
25        guiGoiTinToiTopic("device/" + api_key + "/data", goiTin
           )
26        Cap nhat thoi diem gui cuoi cung
27 Ket thuc Ham

```

Listing 3.2: Giải thuật vòng lặp chính điều phối firmware ESP32

Mã nguồn C++ chi tiết điều phối vòng lặp và logic gửi tin được đính kèm tại Phụ lục A.1.

c. Xử lý tin nhắn cấu hình nhận được

Khi nhận cấu hình từ topic `devices/{mac}/config`, callback MQTT giải mã thông điệp JSON để lưu thông tin định danh thiết bị vào Flash, đặt cờ đăng ký thành công

(provisioned) để chuyển sang chế độ gửi dữ liệu, hoặc thực hiện đặt lại thiết bị (reset) khi có yêu cầu:

```

1 Ham callbackNhanTin(topic, goiTin)
2   Giai ma goi tin JSON nhan duoc
3   Neu goi tin co lenh reset ("action" == "reset"):
4     Mo vung luu tru "device" o che do ghi-doc
5     Xoa toan bo cau hinh luu tru
6     Dong vung luu tru
7     Khoi dong lai thiet bi (ESP.restart())
8     Tra ve
9
10  Neu goi tin chua "device_id" va "api_key":
11    ghiCauHinhVaoFlash(device_id, api_key)
12    Cap nhat bien trang thai: deviceId = device_id,
13                          deviceApiKey = api_key
14    Dat trang thai dang ky thanh cong (provisioned = True)
15 Ket thuc Ham

```

Listing 3.3: Giai thuat callback xu ly thong diep cau hinh tu MQTT Broker

3.2.3 Thiết kế cơ sở dữ liệu

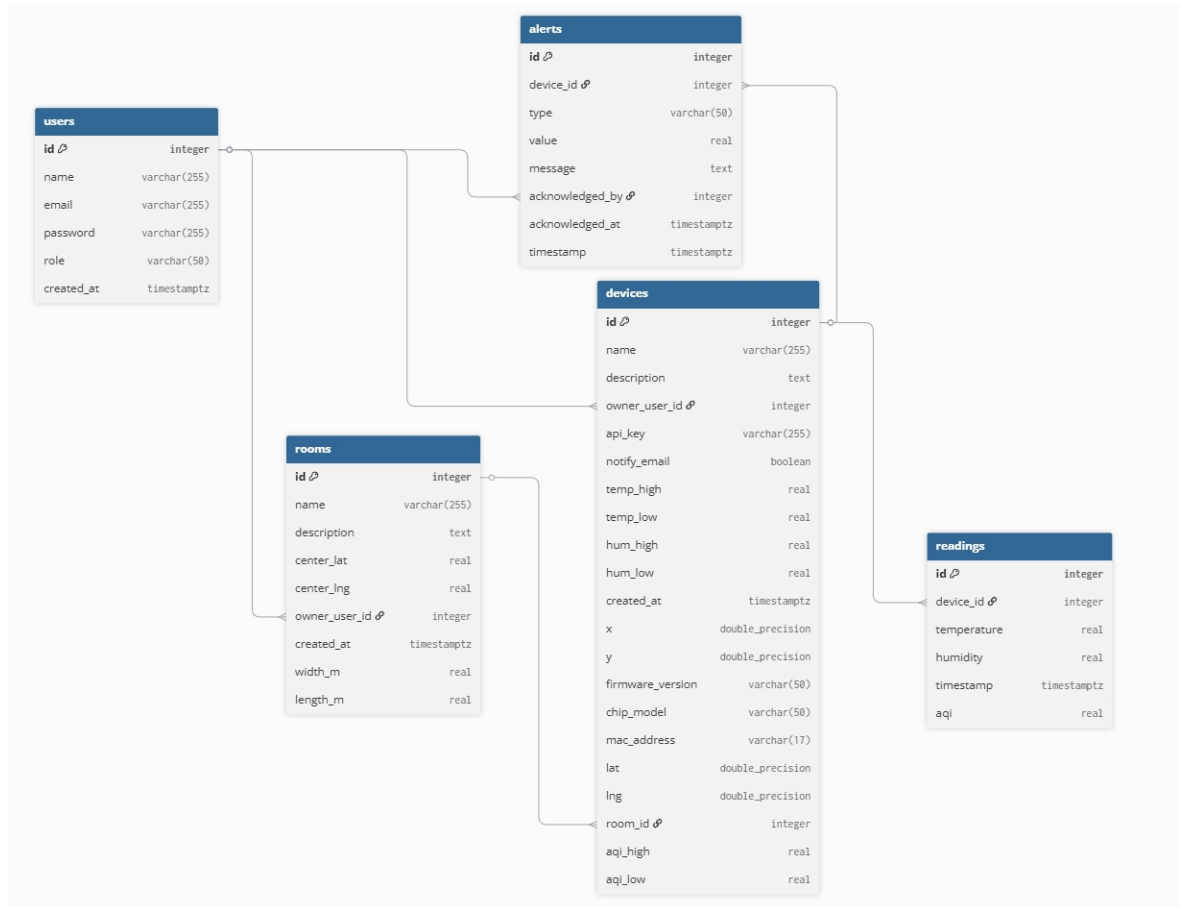
a. Lựa chọn PostgreSQL lưu trữ dữ liệu

Hệ thống lựa chọn hệ quản trị cơ sở dữ liệu quan hệ **PostgreSQL** để lưu trữ toàn bộ dữ liệu người dùng, cấu hình thiết bị và dữ liệu chuỗi thời gian đo đạc được [21]. PostgreSQL được lựa chọn cho đề án vì:

- Hỗ trợ đầy đủ SQL chuẩn, dễ truy vấn dữ liệu lịch sử theo khoảng thời gian với câu lệnh `WHERE timestamp BETWEEN ... AND ...`.
- Tích hợp tốt với Node.js qua thư viện `pg` (PostgreSQL client).
- Hỗ trợ tạo chỉ mục (index) trên cột `timestamp` và `device_id` giúp tăng tốc truy vấn dữ liệu chuỗi thời gian cho các phép toán học sâu và nội suy.
- Miễn phí, phổ biến, có độ ổn định cực cao và dễ triển khai trên môi trường local cũng như VPS.

b. Sơ đồ ERD

Cơ sở dữ liệu PostgreSQL gồm 5 bảng chính, quan hệ được thể hiện trong Hình 3.5.



Hình 3.5: Sơ đồ ERD cơ sở dữ liệu PostgreSQL

c. Định nghĩa schema cơ sở dữ liệu

Cấu trúc các bảng trong cơ sở dữ liệu được định nghĩa chi tiết tại Phụ lục A.2. Mỗi quan hệ giữa các bảng được thiết lập như sau:

- users → devices**: Một người dùng (tài khoản) có thể sở hữu và quản lý nhiều node thiết bị cảm biến khác nhau. Ràng buộc khóa ngoại `owner_user_id` tham chiếu đến bảng `users`.
- rooms → devices**: Một phòng có thể đặt nhiều node cảm biến để tăng mật độ và độ bao phủ không gian phục vụ bài toán nội suy. Mỗi thiết bị được định vị bằng tọa độ tương đối (x, y) trong phòng thông qua khóa ngoại `room_id`.
- devices → readings**: Ràng buộc một-nhiều, lưu trữ lịch sử chuỗi thời gian các chỉ số đo đạc theo từng mã thiết bị. Chỉ mục `idx_readings_device_time` được tạo trên cặp cột (`device_id`, `timestamp` DESC) giúp tăng tốc độ truy vấn 30 mẫu gần nhất phục vụ mô hình dự báo.

- **devices** → **alerts**: Lưu lại lịch sử các sự kiện cảnh báo bất thường khi chỉ số môi trường vượt ngưỡng.

3.2.4 Thiết kế Backend API

a. Cấu trúc và công nghệ của Backend chính

Máy chủ backend chính được xây dựng trên môi trường **Node.js** sử dụng framework **Express** để xây dựng kiến trúc RESTful API phục vụ các tác vụ xác thực, quản lý người dùng, cấu hình thiết bị và trung chuyển dữ liệu [22]. Node.js sử dụng kiến trúc bất đồng bộ hướng sự kiện (event-driven, non-blocking I/O) giúp xử lý hàng ngàn kết nối đồng thời từ client và thiết bị với mức tiêu tốn tài nguyên tối thiểu.

Các thư viện npm chính được sử dụng để phát triển backend bao gồm:

- **aedes + mqtt**: MQTT broker và client tích hợp trực tiếp trong cùng tiến trình Node.js giúp xử lý sự kiện trong cùng vòng lặp sự kiện, tối thiểu hóa độ trễ trung gian.
- **socket.io**: Thư viện WebSocket hỗ trợ giao tiếp hai chiều, realtime để phân phối dữ liệu ngay lập tức tới dashboard của client mà không cần cơ chế polling [23].
- **pg**: PostgreSQL client cho Node.js phục vụ giao tiếp DB.
- **jsonwebtoken + bcrypt**: Xác thực an toàn bằng mã khóa JWT và băm mật khẩu bảo vệ tài khoản.
- **nodemailer**: Dịch vụ gửi email cảnh báo tự động khi các chỉ số đo đạc vượt quá ngưỡng an toàn được cài đặt.

Các API endpoint được phân chia thành ba nhóm chính, phân quyền truy cập thông qua mã xác thực JWT:

- **Quản lý người dùng và xác thực**: Các API `/api/auth/register`, `/api/auth/login` để đăng ký và đăng nhập nhận mã token bảo mật.
- **Quản lý thiết bị và phòng học**: Các API `GET, POST, PUT, DELETE /api/devices` và `/api/rooms` để xem danh sách, đăng ký và cấu hình tọa độ/ngưỡng cảnh báo cho từng node cảm biến.
- **Dữ liệu môi trường, dự báo và cảnh báo**: API `GET /api/data` truy vấn lịch sử đo, API `GET /api/forecast` gọi FastAPI LSTM service để lấy kết quả dự báo thời gian, và API `GET /api/spatial-forecast` gọi FastAPI spatial service lấy dữ liệu lưới nhiệt độ/độ ẩm không gian.

b. Xác thực người dùng bằng JWT

Xác thực phía API sử dụng cơ chế token JWT (JSON Web Token) để đảm bảo tính an toàn cho tài nguyên. Giải thuật trung gian xác thực (middleware) được mô tả qua mã giả dưới đây:

```

1 Ham xacThucTokenJWT(yeuCau, phanHoi, hamTiepTheo)
2     chuoiXacThuc = Lay Authorization tu yeuCau.headers
3     Neu chuoiXacThuc khong ton tai:
4         Tra ve phanHoi voi ma loi 401 ("Missing token")
5
6     token = Trich xuất phân chuỗi sau từ "Bearer <token>"
7     Thu:
8         thôngTinGiaiMa = Giải mã token bằng khóa bí mật JWT_SECRET
9         Gan yeuCau.user = thôngTinGiaiMa
10        Chuyển tiếp đến hamTiepTheo()
11    Neu xảy ra lỗi giải mã:
12        Tra về phản hồi với mã lỗi 403 ("Invalid token")
13 Ket thúc Ham

```

Listing 3.4: Giải thuật middleware xác thực người dùng bằng token JWT

c. Logic tự động phát hiện vượt ngưỡng cảnh báo

Dịch vụ cảnh báo (alertService.js) được khởi chạy bất đồng bộ mỗi khi backend nhận được bản ghi dữ liệu đo đạc mới từ node cảm biến thông qua MQTT. Dịch vụ đối chiếu giá trị đo với các ngưỡng cấu hình lưu trong cơ sở dữ liệu để đưa ra quyết định gửi thông báo qua giao diện và gửi email cảnh báo tự động:

```

1 Ham kiemTraNguongCanhBao(thietBi, duLieuDo)
2     nhietDo = duLieuDo.temperature
3     doAm = duLieuDo.humidity
4     danhSachCanhBao = []
5
6     Neu nhietDo > thietBi.temp_high:
7         Them canh bao co noi dung "Nhiet do vuot nguong cao"
8     Neu nhietDo < thietBi.temp_low:
9         Them canh bao co noi dung "Nhiet do duoi nguong thap"
10    Neu doAm > thietBi.hum_high:
11        Them canh bao co noi dung "Do am vuot nguong cao"
12    Neu doAm < thietBi.hum_low:
13        Them canh bao co noi dung "Do am duoi nguong thap"

```

```

14
15     Voi moi canh bao trong danhSachCanhBao:
16         Ghi ban ghi canh bao vao bang SQL alerts
17         Neu thietBi cau hinh gui email (notify_email = True):
18             Tao noi dung va gui email canh bao thong qua dich vu
19             Nodemailer
Ket thuc Ham

```

Listing 3.5: Giai thuat kiem tra va kich hoat canh bao tu dong

Mã nguồn JavaScript chi tiết của dịch vụ kiểm tra cảnh báo và lưu cơ sở dữ liệu được đính kèm tại Phụ lục A.3.

3.2.5 Thiết kế và huấn luyện mô hình Machine Learning

a. Lựa chọn FastAPI và PyTorch làm Machine Learning Service

Toàn bộ pipeline liên quan đến học máy được tách thành một **microservice độc lập** chạy bằng Python sử dụng framework **FastAPI** [24]. FastAPI hỗ trợ lập trình bất đồng bộ (asyncio), mang lại hiệu năng cao tương đương Node.js và Go, đồng thời tự động tạo giao diện thử nghiệm API thông qua tài liệu OpenAPI.

- **LSTM inference (cổng 8000):** Nhận yêu cầu từ backend Node.js, truy vấn PostgreSQL lấy cửa sổ 30 bản ghi gần nhất, chạy mô hình LSTM (PyTorch) và trả về 15 giá trị dự báo kế tiếp.
- **Nội suy không gian (cổng 8001):** Kết hợp dữ liệu vị trí và giá trị đo đạc thực tế của các cảm biến trong phòng, tính toán nội suy Kriging và IDW để trả về lưới dữ liệu mượt mà kích thước 40×50 điểm làm bản đồ nhiệt.

Việc tách riêng biệt này giúp backend chính không bị nghẽn luồng xử lý do các phép toán học sâu và nội suy nặng. Mô hình Machine Learning được thiết kế và huấn luyện bằng **PyTorch** - thư viện học sâu mạnh mẽ hỗ trợ đồ thị tính toán động linh hoạt [25].

b. Bài toán dự báo chuỗi thời gian và Kiến trúc mô hình LSTM

Bài toán dự báo chuỗi thời gian: Dữ liệu môi trường đo được theo chu kỳ cố định là các biến liên tục tạo thành chuỗi thời gian (time series) [26]. Với cửa sổ quan sát quá khứ $w = 30$ mẫu (2.5 phút) và tầm nhìn dự báo tương lai $h = 15$ mẫu (1.25 phút), đầu vào của mô hình là một ma trận đa biến kích thước 30×3 gồm 3 thông số: nhiệt độ, độ ẩm và AQI.

Mô hình học sâu LSTM được chọn thay thế các mô hình tuyến tính cổ điển (như ARIMA) để giải quyết các mối quan hệ phi tuyến phức tạp trong dữ liệu môi trường.

Kiến trúc LSTM (Long Short-Term Memory): LSTM giải quyết hiện tượng triệt tiêu đạo hàm của RNN truyền thống thông qua cấu trúc Cell State làm nhiệm vụ lưu giữ thông tin dài hạn kết hợp với ba cổng điều khiển chính [16]:

1. **Forget gate** f_t : Quyết định thông tin cũ nào từ Cell State cần được lãng quên:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.1)$$

2. **Input gate** i_t : Quyết định cập nhật thông tin mới nào vào Cell State:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3.3)$$

3. **Output gate** o_t : Quyết định phần giá trị nào của Cell State sẽ được xuất ra làm hidden state tiếp theo:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (3.4)$$

$$h_t = o_t \odot \tanh(C_t) \quad (3.5)$$

Trong đó σ là hàm sigmoid kích hoạt, $\odot$ là phép nhân Hadamard (nhân từng phần tử), W và b đại diện cho ma trận trọng số và bias được tối ưu trong quá trình huấn luyện.

c. Quy trình chuẩn bị dữ liệu và huấn luyện

Dữ liệu lịch sử từ bảng `readings` được tiền xử lý qua các bước: loại bỏ nhiễu/giá trị lỗi (NULL), đưa các thuộc tính về khoảng giá trị $[0, 1]$ bằng bộ chuẩn hoá `MinMaxScaler` của thư viện `scikit-learn` để tăng tốc độ hội tụ khi huấn luyện. Bộ chuẩn hóa được lưu trữ dưới dạng file `scaler.pkl` để sử dụng lại khi suy luận.

Mô hình được huấn luyện bằng hàm tối ưu Adam (learning rate = 10^{-3}), tối ưu hóa hàm mất mát Mean Squared Error (MSELoss). Quy trình chi tiết mã nguồn huấn luyện LSTM được trình bày tại Phụ lục A.4, và quy trình thực hiện API suy luận của FastAPI service được đính kèm tại Phụ lục A.5.

d. Bài toán nội suy không gian và các thuật toán nội suy

Bài toán nội suy không gian: Với n node cảm biến đặt tại các vị trí (lat_i, lng_i) , cần ước lượng giá trị liên tục $\hat{z}$ tại mọi điểm chưa đo (lat, lng) để tạo bản đồ nhiệt phân bố. Hệ thống sử dụng kết hợp hai phương pháp:

- **IDW (Inverse Distance Weighting):** Phương pháp nội suy xác định, ước tính giá trị tại điểm cần đo bằng trung bình có trọng số của các điểm lân cận với trọng số tỷ lệ nghịch với khoảng cách lũy thừa $p = 2$ [18]:

$$\hat{z}(x) = \frac{\sum_{i=1}^n w_i(x) \cdot z_i}{\sum_{i=1}^n w_i(x)}, \quad w_i(x) = \frac{1}{d(x, x_i)^2} \quad (3.6)$$

IDW được sử dụng để sinh ra các **node ảo** tại các góc và cạnh của phòng nhằm tăng độ bao phủ biên, hạn chế sai sót khi số lượng thiết bị thực đo ban đầu còn ít.

- **Ordinary Kriging:** Phương pháp nội suy địa thống kê tối thiểu hóa phương sai ước lượng (BLUE) bằng cách mô hình hóa tương quan không gian qua semivariogram [17]. Kriging sử dụng Variogram dạng Gaussian để tính toán mượt mà giá trị của toàn bộ lưới kích thước 40×50 điểm, hiển thị trực quan bản đồ nhiệt trên Leaflet.

Mã nguồn Python thực thi quá trình kết hợp IDW sinh node góc và nội suy Kriging lưới 40×50 điểm được đính kèm tại Phụ lục A.6.

3.2.6 Thiết kế Dashboard (Frontend)

a. Lựa chọn và mô tả chi tiết công nghệ Frontend

Giao diện Dashboard được xây dựng bằng các công nghệ web hiện đại để đáp ứng tính trực quan và khả năng cập nhật tức thời:

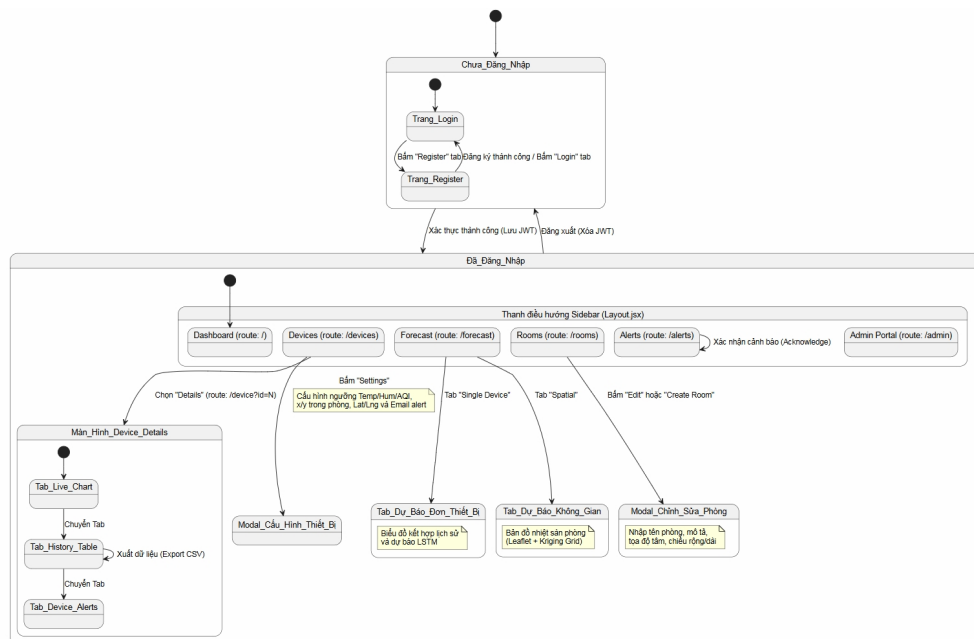
- **React và Vite:** Giao diện Web được xây dựng bằng React 19 theo kiến trúc component giúp phân tách và tái sử dụng các thành phần UI [27]. Vite đóng vai trò làm công cụ bundler, hỗ trợ Hot Module Replacement (HMR) cho tốc độ phản hồi cực nhanh trong quá trình phát triển và tối ưu mã nguồn khi build production. Cơ chế cập nhật State của React giúp giao diện phản ứng tức thời với dữ liệu realtime truyền tới từ WebSockets.
- **Chart.js và react-chartjs-2:** Thư viện Chart.js kết hợp wrapper React được sử dụng để vẽ các biểu đồ chỉ số môi trường theo thời gian. Khi nhận được sự kiện

dữ liệu cảm biến mới từ Socket.io, component tự động cập nhật State, Chart.js thực hiện vẽ thêm điểm dữ liệu mới vào biểu đồ mà không cần tải lại toàn bộ trang web.

- **Leaflet và react-leaflet:** Thư viện bản đồ tương tác mã nguồn mở nhẹ Leaflet được sử dụng để hiển thị mặt bằng phòng học [28]. Lưới dữ liệu 40×50 điểm sau khi nội suy được chuyển thành lớp phủ màu sắc (gradient từ mát mẻ đến nóng ảm) nằm đè lên sơ đồ mặt bằng, hiển thị bản đồ nhiệt trực quan cho người vận hành.

b. Sơ đồ điều hướng các trang

Hình 3.6 mô tả sơ đồ điều hướng giữa các trang trong React dashboard. Trang Login là điểm vào bắt buộc; sau khi xác thực thành công, người dùng được chuyển đến Dashboard chính.



Hình 3.6: Sơ đồ điều hướng các trang trong React dashboard

Các trang chức năng bao gồm:

- **Dashboard (Trang chủ):** Hiển thị danh sách tóm tắt thiết bị, cảnh báo gần nhất, biểu đồ cập nhật trực tiếp dữ liệu đo đạc (realtime line chart).
- **Thiết bị (Devices):** Danh sách chi tiết các node cảm biến, cho phép thay đổi cấu hình ngưỡng cảnh báo và xem lại dữ liệu lịch sử.

- **Dự báo (Forecast):** Tab dự báo trạm (Single Device) hiển thị biểu đồ dự báo LSTM tương lai và Tab dự báo không gian (Spatial) hiển thị heatmap nội suy không gian của căn phòng được chọn.
- **Quản lý phòng (Rooms):** Nơi người quản trị tạo sơ đồ phòng học, định vị tọa độ x, y đặt các thiết bị trên mặt bằng sàn.
- **Cảnh báo (Alerts):** Danh sách lịch sử các cảnh báo kèm chức năng xác nhận đã xử lý (acknowledge).

3.3 Triển khai

3.3.1 Môi trường triển khai

Hệ thống được triển khai và kiểm thử trong môi trường phòng lab với cấu hình cụ thể như sau:

Bảng 3.4: Cấu hình môi trường triển khai

Thành phần	Cấu hình
Máy chủ Backend	Máy tính cá nhân / Laptop
Hệ điều hành	Ubuntu 22.04 LTS / Windows 11 (WSL2)
Node.js	v18 LTS
Python	3.10+
PostgreSQL	v15, chạy local
Node IoT	ESP32 DevKit v1 + DHT11
Mạng	Wi-Fi LAN cùng subnet

3.3.2 Hướng dẫn khởi động hệ thống

Quy trình khởi động hệ thống theo thứ tự:

1. **Khởi động cơ sở dữ liệu PostgreSQL** và thực thi các câu lệnh SQL khởi tạo cấu trúc bảng từ tệp `schema.sql` (chi tiết xem tại Phụ lục A.2).
2. **Khởi động Backend Node.js:** Di chuyển vào thư mục `server/`, cài đặt các gói thư viện phụ thuộc bằng lệnh `npm install`, tạo tệp cấu hình biến môi trường `.env` chứa thông tin kết nối PostgreSQL, khóa bí mật `JWT_SECRET`, và thông tin SMTP email cảnh báo. Cuối cùng khởi chạy máy chủ bằng lệnh `node index.js`.

3. **Khởi động hai FastAPI Machine Learning service (chạy đồng thời):** Di chuyển vào thư mục `forecast/`. Khởi chạy service dự báo chuỗi thời gian LSTM ở cổng 8000 bằng lệnh: `uvicorn time_forecast.main:app -port 8000 -reload`. Khởi chạy service nội suy không gian địa thống kê Kriging ở cổng 8001 bằng lệnh: `uvicorn spatial_forecast:app -port 8001 -reload`.
4. **Khởi động Frontend React:** Di chuyển vào thư mục `frontend/`, chạy lệnh `npm install` để cài đặt các thư viện React. Khởi chạy máy chủ phát triển frontend ở chế độ Local Development bằng lệnh `npm run dev` (mặc định chạy ở cổng 5173).
5. **Nạp firmware lên ESP32:** Mở file `sensor/sensor.ino` trên môi trường phát triển Arduino IDE. Cập nhật các hằng số Wi-Fi SSID, mật khẩu mạng Wi-Fi và địa chỉ IP tĩnh của máy chủ backend làm MQTT Broker. Thực hiện biên dịch và nạp chương trình lên vi điều khiển ESP32 qua cáp micro-USB.
6. **Kiểm thử giao diện:** Mở trình duyệt web truy cập địa chỉ `http://localhost:5173`. Đăng ký tài khoản người dùng đầu tiên để đăng nhập vào Dashboard. Bật nguồn node cảm biến ESP32, thiết bị sẽ tự động đăng ký và truyền dữ liệu cập nhật tức thời lên giao diện biểu đồ realtime của hệ thống.

3.3.3 Script mô phỏng node IoT

Để phục vụ việc kiểm thử khả năng chịu tải, luồng dữ liệu truyền tải của nhiều thiết bị đo cùng lúc mà không cần phần cứng thực tế, đồ án cung cấp một script mô phỏng chạy bằng Node.js. Thuật toán của script mô phỏng được mô tả bằng mã giả như sau:

```

1 Ham moPhongGuiDuLieu()
2   Ket noi toi dia chi MQTT Broker noi bo ("mqtt://localhost:1883"
3   )
4   Dat ma API_KEY = "your-api-key-here" (ma khoa xac thuc da dang
5   ky trong DB)
6   Dinh ky moi 5 giay thuc hien:
7     nhietDo = Sinh so thuc ngau nhien tu 20.0 den 35.0 (lay 1
8     chu so thap phan)
9     doAm = Sinh so thuc ngau nhien tu 40.0 den 80.0
10    aqi = Sinh so thuc ngau nhien tu 30.0 den 100.0 (chi so
11    chat luong khong khi gia lap)
12    goiTin = Dong goi JSON { "temperature": nhietDo, "humidity"
13    : doAm, "aqi": aqi }
```

```

9       Publish gọi tin lên topic "device/" + API_KEY + "/data"
10      In thông tin log ra màn hình console
11      Kết thúc hàm

```

Listing 3.6: Giải thuật script mô phỏng node thiết bị gửi dữ liệu lên máy chủ

3.4 Tổng hợp công nghệ sử dụng trong hệ thống

Bảng 3.5 tổng hợp toàn bộ các công nghệ cốt lõi được lựa chọn sử dụng xuyên suốt các thành phần của hệ thống cùng với các lý do kỹ thuật tương ứng.

Bảng 3.5: Tổng hợp công nghệ sử dụng trong hệ thống

Thành phần	Công nghệ	Lý do lựa chọn
Vi điều khiển	ESP32	Tích hợp Wi-Fi, chi phí thấp, hiệu năng xử lý cao, hệ sinh thái phong phú
Cảm biến	DHT11	Giao tiếp đơn giản 1-Wire, chi phí cực thấp, đủ chính xác cho prototype
Giao thức IoT	MQTT (Aedes)	Gói tin nhẹ, overhead tối thiểu, kết nối TCP bền vững, tích hợp Node.js
Backend chính	Node.js/Express	Xử lý bất đồng bộ I/O tốt, điều phối nhiều kết nối realtime đồng thời
Realtime push	Socket.io	Thiết lập WebSocket hai chiều nhanh, quản lý room theo thiết bị hiệu quả
Cơ sở dữ liệu	PostgreSQL	Hệ quản trị quan hệ bền vững, truy vấn nhanh chuỗi thời gian nhờ lập chỉ mục
ML Framework	PyTorch	Hỗ trợ đồ thị tính toán động, dễ xây dựng và lưu model weights cho suy luận
ML Service	FastAPI (Python)	Lập trình async Python API hiệu năng cao, tự sinh OpenAPI docs tiện lợi
Nội suy không gian	Kriging / IDW	Kriging tối ưu phương sai nội suy mịn, IDW hỗ trợ góc biên phòng
Frontend	React 19 + Vite	UI dạng component dễ bảo trì, Vite HMR cập nhật tức thời khi dev
Biểu đồ	Chart.js	Dung lượng nhẹ, hỗ trợ render động khi có dữ liệu mới qua WebSockets
Bản đồ mặt bằng	Leaflet	Thư viện bản đồ tương tác mã nguồn mở nhẹ, dễ vẽ lớp heatmap tùy biến

CHƯƠNG 4.

ĐÁNH GIÁ HỆ THỐNG VÀ CÁC GIẢI PHÁP NỔI BẬT

Chương này trình bày các kết quả đánh giá thực tế của toàn bộ hệ thống AIoT giám sát và dự báo chất lượng không khí. Nội dung bao gồm kết quả xây dựng các thành phần giao diện chức năng, quy trình kiểm thử hộp đen để xác thực chất lượng phần mềm, cùng các phân tích chuyên sâu về ba giải pháp kỹ thuật nổi bật của đề án: pipeline dự báo thời gian thực sử dụng mạng học sâu LSTM, mô hình nội suy địa thống kê Kriging/IDW tạo bản đồ nhiệt phân bố, và tối ưu hóa hạ tầng giao thức truyền tin MQTT kết hợp WebSockets.

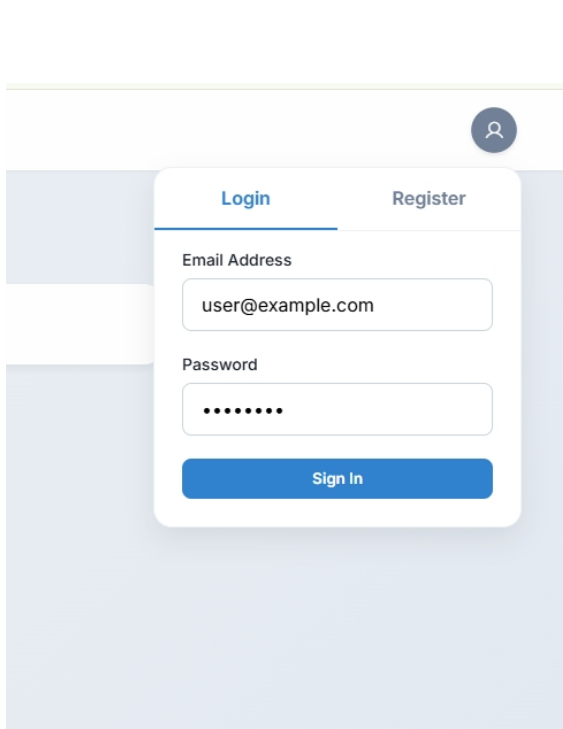
4.1 Kết quả và đánh giá hệ thống

4.1.1 Kết quả đạt được

Sau quá trình thiết kế và triển khai, đề án đã xây dựng thành công hệ thống hoàn chỉnh với đầy đủ các thành phần hoạt động ổn định. Kết quả cụ thể từng thành phần:

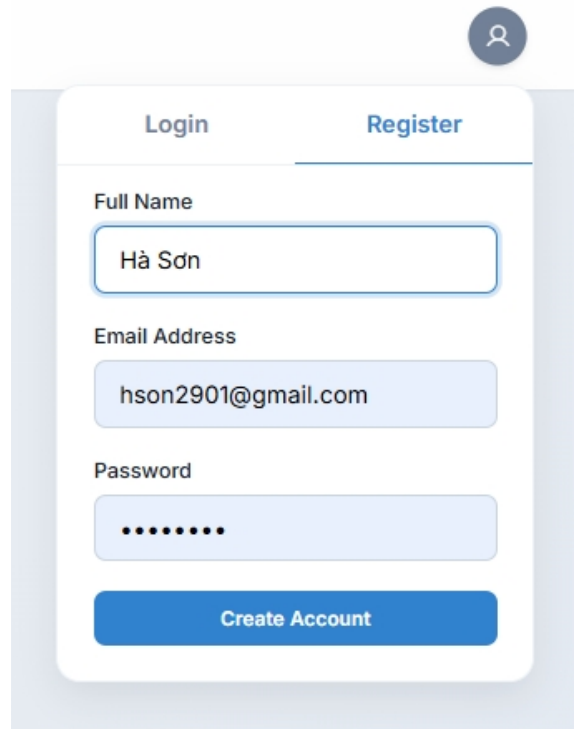
- **Node cảm biến:** Hoạt động ổn định, đọc và gửi dữ liệu định kỳ 5 giây qua MQTT. Cơ chế tự động kết nối lại (reconnect) đảm bảo node phục hồi kết nối sau khi mất mạng không cần can thiệp thủ công.
- **Backend điều phối:** Tiếp nhận và xử lý dữ liệu từ nhiều node đồng thời. MQTT broker tích hợp hoạt động ổn định trong cùng tiến trình với API Node.js. Máy chủ phản hồi trung bình dưới 100ms cho các truy vấn dữ liệu thông thường.
- **Dịch vụ Machine Learning:** Suy luận LSTM hoàn thành trong dưới 200ms cho mỗi node cảm biến (mô hình lookback 30, horizon 15). Thuật toán Kriging tính toán lưới 40×50 điểm trong khoảng 1–2 giây tùy thuộc số lượng điểm đo đầu vào.
- **Giao diện Web Dashboard:** Biểu đồ cập nhật tức thời khi nhận sự kiện truyền tải thông qua WebSockets, không cần tải lại trang. Bản đồ nhiệt (heatmap) phân bố không gian hiển thị trực quan và tương thích tốt trên các trình duyệt hiện đại.
- **Mô hình học sâu LSTM:** Huấn luyện hội tụ tốt trên tập dữ liệu đo đạc thực nghiệm. Đường cong suy hao (loss curve) thể hiện xu hướng giảm đều, ổn định và không xảy ra hiện tượng quá khớp (overfitting).

Để trực quan hoá các kết quả đạt được, toàn bộ giao diện chức năng chính của hệ thống được minh họa qua ba cụm hình ảnh giao diện dưới đây:



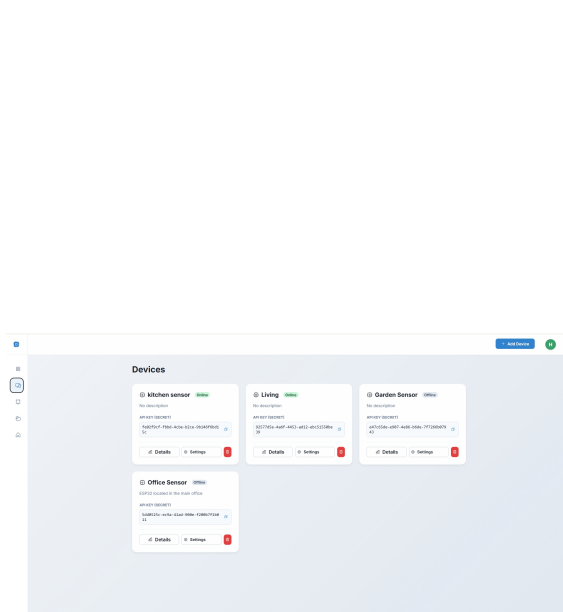
The image shows a mobile application interface with a 'Login' and 'Register' toggle. The 'Login' tab is active, showing fields for 'Email Address' (with the placeholder 'user@example.com') and 'Password' (with masked dots). A blue 'Sign In' button is at the bottom. A user profile icon is in the top right corner.

(a) Màn hình đăng nhập



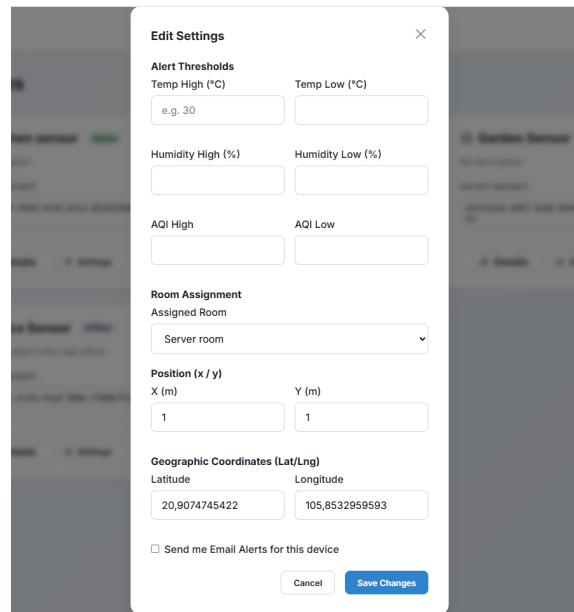
The image shows the 'Register' tab of the mobile application. It contains fields for 'Full Name' (with the placeholder 'Hà Sơn'), 'Email Address' (with the placeholder 'hson2901@gmail.com'), and 'Password' (with masked dots). A blue 'Create Account' button is at the bottom. A user profile icon is in the top right corner.

(b) Màn hình đăng ký



The image shows a 'Devices' management screen. It lists four sensors: 'Kitchen sensor', 'Living', 'Garden Sensor', and 'Office Sensor'. Each sensor card displays its name, status (e.g., 'Online'), and a 'Details' button. A 'Settings' button is also visible at the bottom right of the list.

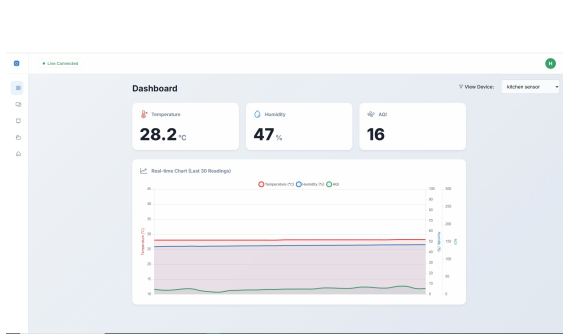
(c) Danh sách thiết bị IoT



The image shows an 'Edit Settings' modal window. It includes sections for 'Alert Thresholds' (Temp High/Low, Humidity High/Low, AQI High/Low), 'Room Assignment' (Assigned Room: 'Server room'), 'Position (x / y)' (X: 1, Y: 1), and 'Geographic Coordinates (Lat/Lng)' (Latitude: 20,9074745422, Longitude: 105,8532959593). There is a checkbox for 'Send me Email Alerts for this device' and 'Save Changes' and 'Cancel' buttons at the bottom.

(d) Cấu hình ngưỡng cảnh báo

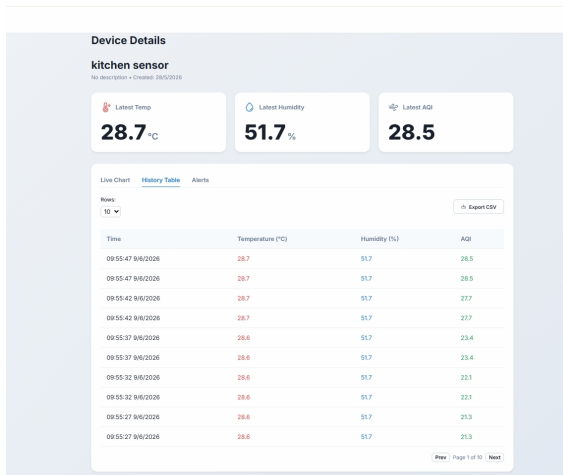
Hình 4.1: Giao diện chức năng hệ thống — Đăng nhập, đăng ký và quản lý thiết bị



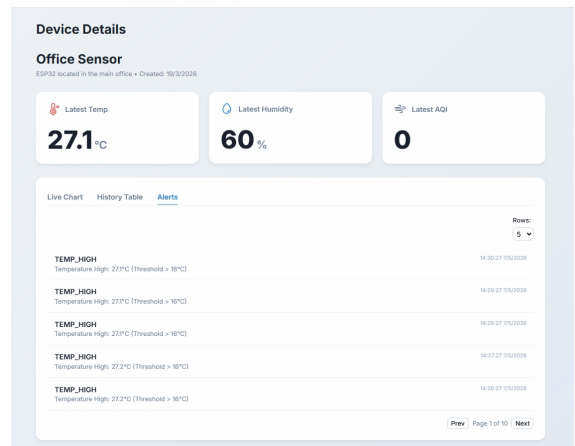
(a) Dashboard tổng quan



(b) Biểu đồ chi tiết thiết bị (Live Chart)

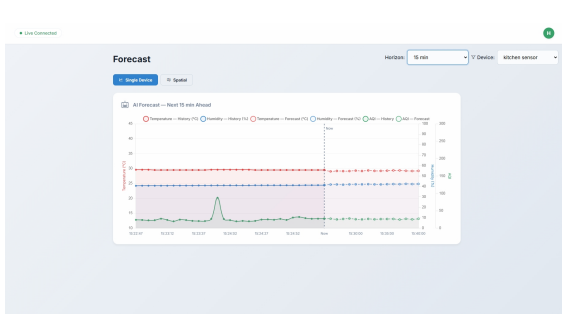


(c) Lịch sử dữ liệu cảm biến

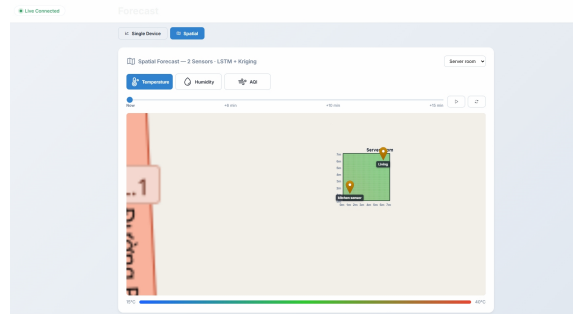


(d) Lịch sử cảnh báo của thiết bị

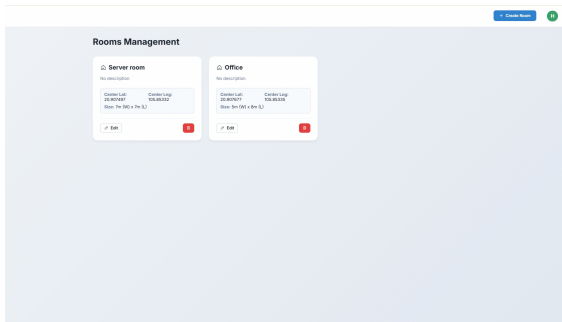
Hình 4.2: Giao diện chức năng hệ thống — Dashboard và các tab chi tiết thiết bị



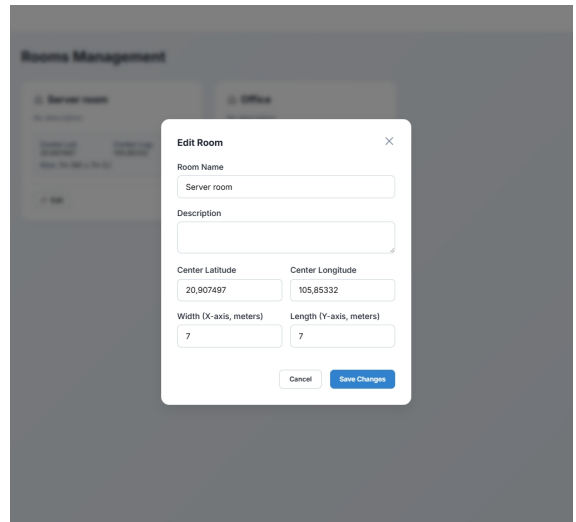
(a) Dự báo chuỗi thời gian LSTM



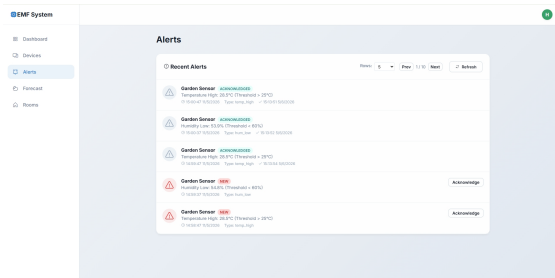
(b) Heatmap không gian



(c) Quản lý danh sách phòng



(d) Cấu hình kích thước và vị trí node



(e) Danh sách cảnh báo hệ thống

Hình 4.3: Giao diện dự báo thời gian/không gian, quản lý phòng và cảnh báo hệ thống

4.1.2 Minh họa các chức năng chính

Chức năng 1 — Đăng ký node IoT tự động: Khi node cảm biến khởi động lần đầu và kết nối mạng thành công, hệ thống tự động cấp phát mã khoá API để định danh mà không cần can thiệp thủ công của người dùng. Node sẵn sàng gửi dữ liệu môi trường trong vòng vài giây.

Chức năng 2 — Giám sát thời gian thực: Các biểu đồ chỉ số đo đạc trên giao diện dashboard cập nhật tức thời theo tần suất gửi dữ liệu của thiết bị đo. Người dùng có thể giám

sát đồng thời hoạt động của nhiều node.

Chức năng 3 — Dự báo môi trường tương lai: Khi người dùng yêu cầu xem dự báo, hệ thống hiển thị kết quả dự báo 15 bước tiếp theo ở dạng biểu đồ kết hợp (phân biệt bằng màu sắc khác để người dùng dễ nhận diện xu hướng biến động sắp tới).

Chức năng 4 — Bản đồ nhiệt không gian phòng học: Từ dữ liệu đo đạc phân tán của các cảm biến trong phòng, hệ thống nội suy và render lớp heatmap màu sắc trực quan (từ màu lạnh mát mẻ đến màu nóng ấm áp) phủ trực tiếp lên sơ đồ phòng học.

Chức năng 5 — Cảnh báo tự động: Khi phát hiện bất kỳ chỉ số nào vượt khỏi ngưỡng cấu hình an toàn, hệ thống ngay lập tức tạo bản ghi cảnh báo, hiển thị pop-up thông báo và tự động gửi email cảnh báo chi tiết tới tài khoản người quản lý.

4.1.3 Kiểm thử hệ thống

Chúng tôi tiến hành kiểm thử các chức năng chính theo phương pháp hộp đen (black-box testing) để kiểm nghiệm và đảm bảo chất lượng, độ tin cậy của phần mềm. Kết quả kiểm thử được mô tả cụ thể trong bốn bảng dưới đây:

Bảng 4.1: Kiểm thử chức năng đăng ký và kết nối node IoT

Trường hợp kiểm thử	Kết quả mong đợi	Kết quả
Node mới kết nối lần đầu	Máy chủ tạo thiết bị mới, cấp mã khóa API và cấu hình qua MQTT	Đạt
Node cũ kết nối lại	Máy chủ xác thực và trả về cấu hình hiện tại, không tạo bản ghi mới	Đạt
Wi-Fi ngắt đột ngột	Node tự động reconnection định kỳ mỗi 5 giây cho đến khi có mạng	Đạt
Mã API không hợp lệ	Máy chủ bỏ qua thông điệp dữ liệu, ghi log cảnh báo xâm nhập	Đạt

Bảng 4.2: Kiểm thử chức năng thu thập và hiển thị dữ liệu realtime

Trường hợp kiểm thử	Kết quả mong đợi	Kết quả
Node gửi gói tin đo đặc	Biểu đồ dashboard nhận sự kiện và vẽ lại trong vòng dưới 2 giây	Đạt
Cảm biến lỗi (trả về NaN)	Node bỏ qua chu kỳ gửi lỗi, tiếp tục đo đặc chu kỳ tiếp theo	Đạt
Nhiều node gửi tin đồng thời	Máy chủ tiếp nhận song song và ghi nhận đầy đủ, không nghẽn/mất tin	Đạt
Client mở nhiều tab trình duyệt	Mỗi tab nhận và hiển thị đúng luồng dữ liệu của node tương ứng	Đạt

Bảng 4.3: Kiểm thử chức năng dự báo và heatmap không gian

Trường hợp kiểm thử	Kết quả mong đợi	Kết quả
Có đủ 30 mẫu lịch sử	Dịch vụ dự báo chạy suy luận, trả về đủ 15 bước tương lai	Đạt
Có ít hơn 30 mẫu lịch sử	Trả về mã lỗi 400 kèm hướng dẫn chờ tích lũy đủ dữ liệu quan sát	Đạt
Service Machine Learning bị tắt	Máy chủ backend trả về lỗi 503 thay vì crash luồng chạy chính	Đạt
Vẽ heatmap khi chỉ có 1 node	Hệ thống tự động sinh 4 node góc bù biên bằng IDW và vẽ heatmap	Đạt

Bảng 4.4: Kiểm thử chức năng cảnh báo vượt ngưỡng

Trường hợp kiểm thử	Kết quả mong đợi	Kết quả
Nhiệt độ vượt ngưỡng cao	Tạo sự kiện cảnh báo trong DB, gửi email tới người dùng trong 30 giây	Đạt
Độ ẩm dưới ngưỡng thấp	Tạo sự kiện cảnh báo, hiển thị pop-up đỏ tức thời trên dashboard	Đạt
Xác nhận cảnh báo (Acknowledge)	Đánh dấu sự kiện đã xử lý, không gửi email lặp lại	Đạt
Tắt thông báo email thiết bị	Cảnh báo vẫn được tạo ghi nhận trong DB nhưng không gửi email	Đạt

4.2 Pipeline dự báo chuỗi thời gian LSTM qua FastAPI

4.2.1 Đặt vấn đề

Dữ liệu nhiệt độ, độ ẩm và chất lượng không khí trong phòng biến động liên tục và chịu ảnh hưởng trực tiếp của hoạt động con người hoặc điều kiện thời tiết bên ngoài. Việc dự báo sớm các chỉ số này mang ý nghĩa lớn cho việc vận hành điều hòa, thông gió chủ động. Thách thức kỹ thuật đặt ra:

- **Tính phi tuyến:** Sự biến đổi của chỉ số môi trường trong phòng học/văn phòng có quán tính nhiệt lớn và chịu ảnh hưởng của nhiều yếu tố hỗn hợp, khiến các mô hình thống kê cổ điển như ARIMA khó đạt độ chính xác cao.
- **Ràng buộc tài nguyên phía máy chủ:** Huấn luyện và chạy suy luận mô hình Deep Learning đòi hỏi tài nguyên CPU/GPU lớn, dễ gây nghẽn luồng xử lý chính của backend Node.js.
- **Độ trễ phản hồi:** Mô hình dự báo cần cung cấp kết quả suy luận tức thời dưới 500ms khi có yêu cầu từ client.

4.2.2 Giải pháp

Chúng tôi giải quyết bài toán bằng cách thiết kế mô hình học sâu LSTM đa biến (multi-variate) tích hợp trong một microservice Python (chạy FastAPI) độc lập kết nối với backend Node.js.

Giải thuật quy trình huấn luyện mô hình LSTM được mô tả qua mã giả dưới đây:

1. Tải toàn bộ chuỗi dữ liệu đặc trưng (nhiệt độ, độ ẩm, AQI) tăng dần theo thời gian từ cơ sở dữ liệu PostgreSQL.
2. Tiền xử lý dữ liệu: Loại bỏ nhiễu và chuẩn hóa các đặc trưng về khoảng $[0, 1]$ bằng MinMaxScaler. Lưu tham số scaler vào **file** scaler.pkl.
3. Tạo tập chuỗi dữ liệu trượt trích xuất (Sliding Window):
 - Đầu vào (X): 30 mẫu dữ liệu quá khứ gần nhất (Lookback window = 30).
 - Đầu ra (y): 15 mẫu dữ liệu tương lai cần dự báo (Forecast horizon = 15).
4. Định nghĩa kiến trúc mô hình học sâu Stacked LSTM (2 lớp ẩn gồm 64 units mỗi lớp, bổ sung lớp Dropout = 0.3 ở giữa các tầng và lớp Fully Connected ở đầu ra).

- ```

7 5. Huan luyen mo hinh su dung thuat toan toi uu Adam (learning rate
 = 0.001) dua tren ham mat mat Mean Squared Error (MSE Loss) qua
 so epoch toi da = 100.
8 6. Ap dung co che dung som (Early Stopping) neu loss kiem thu khong
 cai thien sau 10 epoch lien tiep.
9 7. Luu cac trong so toi uu thu duoc vao tep lstm_model.pth.

```

**Listing 4.1:** Giai thuat huan luyen mohinh du bao LSTM da bien

\*Mã nguồn huấn luyện chi tiết bằng PyTorch được đính kèm tại Phụ lục A.4.\*

Khi người dùng gửi yêu cầu dự báo từ dashboard, backend trung chuyển yêu cầu tới API /forecast của FastAPI service để suy luận kết quả theo giải thuật sau:

- ```

1 Khi nhan duoc yeu cau HTTP GET tu client kem device_id:
2   1. Truy van co so du lieu PostgreSQL lay dung 30 ban ghi do gan
   nhat cua thiet bi do.
3   2. Neu so luong ban ghi do < 30: Tra ve ma loi 400 kem thông
   bao chua du du lieu quan sat.
4   3. Nap bo chuan hoa tu tep scaler.pkl va chuyen doi 30 mau du
   lieu do ve khoang [0, 1].
5   4. Chuyen doi du lieu da chuan hoa thanh Tensor dau vao co kích
   thuc (1, 30, 3).
6   5. Chuyen mo hinh LSTM ve che do suy luan (model.eval() va
   torch.no_grad() de giai phong bo nho).
7   6. Dua Tensor qua mo hinh de nhan ket qua dau ra dang vector.
8   7. Su dung bo MinMaxScaler de bien doi nguoc ket qua du bao dau
   ra ve cac chi so vat ly thuc te ban dau.
9   8. Tra ket qua du bao dang mang JSON gom 15 gia tri tiep theo
   ve cho backend.

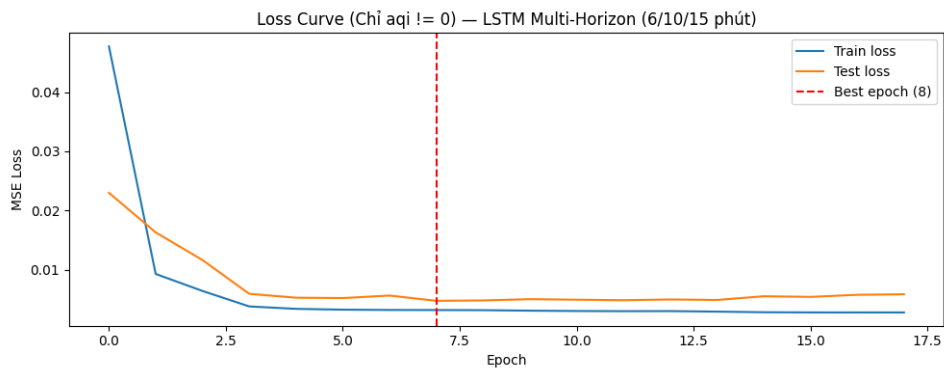
```

Listing 4.2: Giai thuat API phục vụ suy luận du bao của FastAPI service

Mã nguồn FastAPI thực thi logic suy luận dự báo được đính kèm tại Phụ lục A.5.

4.2.3 Đánh giá độ chính xác mô hình

Chúng tôi tiến hành huấn luyện thử nghiệm mô hình trên tập dữ liệu đo đặc thực nghiệm thu được để làm baseline đánh giá ban đầu. Loss curve trong Hình 4.4 cho thấy sự hội tụ nhanh chóng của mô hình mà không gặp hiện tượng quá khớp nhờ có lớp Dropout 0.3 và dừng sớm.



Hình 4.4: Đường cong loss trong quá trình huấn luyện mô hình LSTM

Độ chính xác của mô hình dự báo chuỗi thời gian được đánh giá thông qua sai số tuyệt đối trung bình (Mean Absolute Error — MAE) tại các mốc thời gian dự báo tương lai cụ thể, chi tiết trong Bảng 4.5:

Bảng 4.5: Sai số MAE của mô hình LSTM theo từng mốc dự báo

Mốc dự báo	MAE nhiệt độ (°C)	MAE độ ẩm (%)	MAE AQI
6 phút	0.216	0.910	4.854
10 phút	0.244	0.880	4.921
15 phút	0.278	0.921	5.012

Kết quả thực nghiệm cho thấy mô hình đạt độ chính xác tương đối cao trên dữ liệu môi trường, sai số nhiệt độ trung bình dưới 0.3°C và sai số độ ẩm dưới 1%. Chỉ số AQI có sai số dao động quanh mức 5 đơn vị. Kết quả sai số có xu hướng tăng nhẹ khi khoảng cách dự báo càng xa (từ 6 phút lên 15 phút), phù hợp hoàn toàn với đặc tính bất định tăng dần theo thời gian của các bài toán dự báo chuỗi thời gian. Kết quả này chứng minh tính khả thi của mô hình trong việc đưa ra các cảnh báo xu hướng sớm cho người dùng.

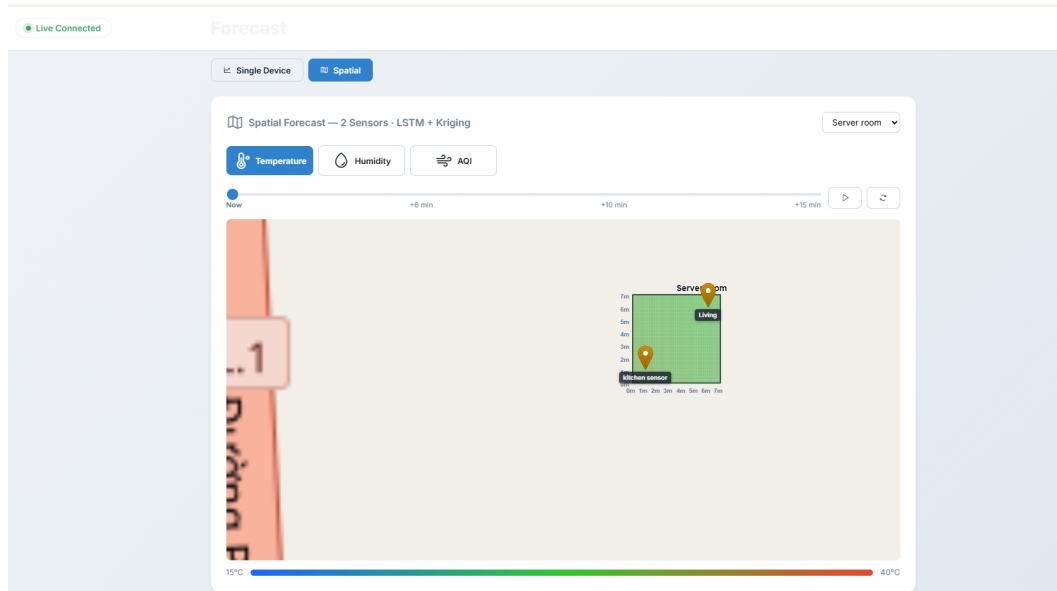
4.3 Nội suy không gian Kriging/IDW để tạo heatmap phân bố

4.3.1 Đặt vấn đề

Việc đặt cảm biến dày đặc khắp phòng học hoặc văn phòng là không khả thi do chi phí và hạ tầng mạng. Tuy nhiên, người vận hành cần giám sát các chỉ số tại **bất kỳ điểm nào** trong phòng để xác định các khu vực tụ nhiệt hoặc thiếu thông gió. Việc áp dụng các thuật toán nội suy thông thường lên số lượng node cảm biến ít thường gây ra hiện tượng méo dữ liệu tại các góc phòng xa cảm biến hoặc ngoại suy (extrapolation) mất kiểm soát ở biên.

4.3.2 Giải pháp

Đồ án đề xuất giải pháp nội suy kết hợp hai tầng: **Sử dụng IDW để sinh node ảo góc phòng** và **Sử dụng Ordinary Kriging để nội suy mịn toàn bộ lưới**.



Hình 4.5: Heatmap phân bố môi trường không gian được tạo từ nội suy Kriging, hiển thị trên bản đồ Leaflet

Giải thuật chi tiết của quy trình nội suy không gian được mô tả qua mã giả như sau:

```

1 Ham tinhToanBanDoNhiet(danhSachNodeThuc, thôngTinPhong)
2   1. Xác định tọa độ tương đối của 4 góc căn phòng dựa trên chiều
   dài và chiều rộng phòng.
3   2. Với mỗi góc phòng:
4       Tính toán khoảng cách từ tất cả các node cảm biến thực
       te.
5       Ước tính giá trị nhiệt độ, độ ẩm tại góc phòng bằng công
       thức IDW (trong số nghịch đảo bình phương khoảng cách).
6       Tạo một node cảm biến ảo đặt tại góc phòng với giá trị
       vừa tính.
7   3. Tập hợp all_nodes = danhSachNodeThuc + 4 node ảo góc phòng.
8   4. Trích xuất danh sách tọa độ X, Y và danh sách giá trị nhiệt
       độ, độ ẩm tương ứng của all_nodes.
9   5. Thiết lập lưới mục tiêu gồm 40 hàng và 50 cột (lưới 40x50 =
       2000 điểm) trải đều diện tích phòng.
10  6. Khởi tạo mô hình OrdinaryKriging cho nhiệt độ với cấu trúc
       Variogram dạng Gaussian.
11  7. Khởi tạo mô hình OrdinaryKriging cho độ ẩm với cấu trúc

```

```

12     Variogram dang Gaussian.
13     8. Thuc thi Kriging noi suy gia tri nhiet do va do am cho luoi
        muc tieu 40x50.
14     9. Tra ve mang toa do luoi cung cac ma tran nhiet do, do am noi
        suy tuong ung.
15 Ket thuc Ham

```

Listing 4.3: Giai thuat noi suy khong gian dia thong ke Kriging/IDW

Mã nguồn Python chi tiết của thuật toán Kriging tích hợp IDW được đính kèm tại Phụ lục A.6.

Kết quả lưới 2000 điểm nội suy được gửi về frontend dưới dạng JSON để giao diện dashboard hiển thị lớp heatmap mượt mà như minh họa ở Hình 4.5, giúp trực quan hóa phân bố không gian dễ dàng.

4.4 Kiến trúc truyền thông MQTT kết hợp Socket.io đảm bảo độ trễ thấp

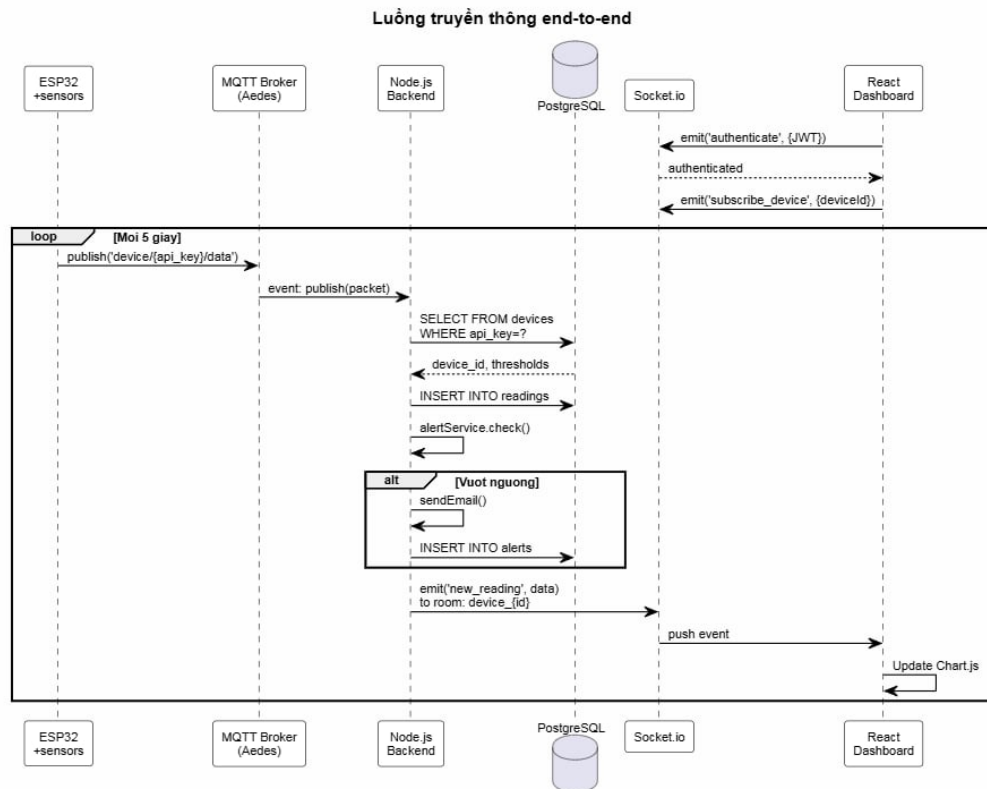
4.4.1 Đặt vấn đề

Hệ thống quan trắc thời gian thực đặt ra yêu cầu nghiêm ngặt về độ trễ truyền dữ liệu end-to-end: từ khi node cảm biến đọc giá trị đến khi biểu đồ trên Web Dashboard được vẽ lại phải dưới 2 giây. Các tiếp cận truyền thống sử dụng HTTP POST gửi tin và HTTP Polling phía Client để lấy tin gây ra các hạn chế:

- **Độ trễ lớn:** HTTP Polling tạo ra độ trễ tỷ lệ thuận với chu kỳ polling. Polling quá dày gây tải nặng cho máy chủ, quá thưa dẫn đến thông tin hiển thị bị trễ.
- **Overhead gói tin lớn:** Giao thức HTTP yêu cầu bắt tay TCP và gửi kèm các header gói tin kích thước lớn (500 bytes mỗi lần gửi), gây lãng phí tài nguyên mạng đối với thiết bị vi điều khiển IoT.

4.4.2 Giải pháp

Hệ thống áp dụng chiến lược kết hợp bổ trợ hai giao thức truyền tin thời gian thực: ****MQTT**** từ node IoT lên máy chủ backend, và ****Socket.io**** (WebSockets) từ máy chủ backend đẩy xuống client dashboard.



Hình 4.6: Luồng truyền thông end-to-end của hệ thống

Toàn bộ luồng dữ liệu end-to-end được minh họa chi tiết trong Hình 4.6. Quy trình gửi dữ liệu MQTT từ vi điều khiển được mô tả qua mã giả như sau:

```

1 Ham guiDuLieuDo(nhietDo, doAm, aqi)
2   Dong goi du lieu thanh chuoi JSON:
3   {
4     "temperature": nhietDo,
5     "humidity": doAm,
6     "aqi": aqi
7   }
8   Dinh nghia topic truyền tin: "device/" + api_key + "/data"
9   Gui thông điệp JSON đã đóng gói lên topic trên máy chủ MQTT
   Broker
10 Ket thuc Ham
  
```

Listing 4.4: Giai thuật đóng gói và gửi dữ liệu đo qua MQTT

Phía máy chủ backend, Broker MQTT tích hợp (Aedes) đón nhận gói tin đo đạc trên sự kiện publish, thực hiện kiểm tra bảo mật, ghi nhận vào cơ sở dữ liệu và đẩy xuống các client đang theo dõi tức thì:

1 Khi Broker nhận được một thông điệp được Publish lên hệ thống:

```

2   Neu topic khong bat dau bang "device/": Tra ve (bo qua goi tin)
3   Trich xuat api_key tu topic truyen tin.
4   Truy van PostgreSQL de kiem tra su ton tai va tinh hop le cua
    api_key:
5       Neu api_key khong hop le: Huy bo xu ly goi tin va ghi nhan
        log canh bao.
6   Giai ma chuoai payload JSON nhan duoc de lay gia tri: nhiet do,
    do am va AQI.
7   Thuc thi truy van SQL chen ban ghi moi vao bang readings.
8   Goi alertService kiem tra cac nguong nhiet do/do am cai dat cua
    thiet bi de kích hoạt canh bao neu can.
9   Xac dinh room WebSocket tuong ung voi thiet bi (ten room: "
    device_" + device_id).
10  Emit su kien "new_reading" toi toan bo client dang theo doi
    room do kem goi tin chua du lieu moi va timestamp hien tai.
11  Ket thuc Ham

```

Listing 4.5: Giai thuat tiep nhan du lieu va dinh tuyen WebSockets tren Backend

Phía giao diện người dùng, trình duyệt duy trì kết nối WebSocket liên tục, nhận dữ liệu mới gửi xuống và cập nhật tức thì trạng thái của giao diện:

```

1  Ham khaiTaoKetNoiRealtime()
2      Thiet lap ket noi WebSockets toi may chu Backend.
3      Gui su kien "authenticate" kem JWT token de xac thuc quyen truy
    cap.
4      Gui su kien "subscribe_device" kem deviceId cua node cam bien
    dang chon de dang ky room.
5      Lang nghe su kien "new_reading" tu may chu gui xuong:
6          Doc thong tin nhiet do, do am, AQI va thoi diem nhan tin.
7          Them gia tri moi vao mang du lieu cua bieu do Chart.js.
8          Cap nhat React State de kích hoạt qua trình tu dong ve lai
    giao dien bieu do.
9  Ket thuc Ham

```

Listing 4.6: Giai thuat tiep nhan du lieu thoi gian thuc phía Client-side

Sự kết hợp này mang lại độ trễ truyền dữ liệu end-to-end cực thấp (trung bình dưới 500ms), hoàn toàn loại bỏ cơ chế polling lãng phí và đảm bảo tính đồng bộ thời gian thực cho toàn hệ thống.

CHƯƠNG 5.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết quá trình thực hiện đồ án, đánh giá những thành tựu đạt được so với mục tiêu ban đầu và nhận diện các hạn chế còn tồn tại. Cuối cùng, chương đề xuất các hướng phát triển tiềm năng để nâng cao chất lượng và mở rộng phạm vi ứng dụng của hệ thống trong tương lai.

5.1 Kết luận

Trước thực trạng các hệ thống quan trắc môi trường hiện tại hoặc có chi phí quá cao, hoặc thiếu khả năng dự báo và trực quan hóa không gian, đồ án đã đề xuất và triển khai thành công một **hệ thống AIoT giám sát và dự báo chất lượng không khí** hoàn chỉnh với chi phí thấp và mã nguồn mở.

5.1.1 Kết quả đạt được

Về mặt kỹ thuật, hệ thống đã đạt được các thành tựu cụ thể sau:

- **Node cảm biến IoT:** Các node cảm biến hoạt động ổn định, thu thập và gửi dữ liệu định kỳ 5 giây qua MQTT với cơ chế tự động đăng ký (provisioning) và khôi phục kết nối (reconnect) không cần can thiệp thủ công.
- **Backend xử lý thời gian thực:** Kiến trúc backend chính kết hợp MQTT broker và truyền tải WebSockets hoạt động ổn định, phân phối dữ liệu đến dashboard với độ trễ end-to-end trung bình dưới 500ms trong điều kiện mạng LAN.
- **Pipeline dự báo LSTM:** Mô hình LSTM với cửa sổ quan sát 30 và tầm nhìn dự báo 15 phút được huấn luyện hội tụ tốt, phục vụ suy luận qua microservice độc lập trong dưới 200ms mỗi lần dự báo.
- **Nội suy không gian Kriging/IDW:** Từ n điểm đo rời rạc, hệ thống tái tạo bản đồ phân bố nhiệt độ, độ ẩm và chất lượng không khí liên tục trên lưới 40×50 điểm, render heatmap trực quan trên Leaflet trong dưới 2 giây.
- **Dashboard giám sát:** Giao diện người dùng hiển thị dữ liệu realtime, biểu đồ lịch sử, kết quả dự báo và heatmap không gian trên cùng một nền tảng web, không cần tải lại trang.

- **Chi phí thấp:** Tổng chi phí phần cứng một node cảm biến dưới 150.000 VND, thấp hơn nhiều lần so với các thiết bị thương mại tương đương, phù hợp triển khai mật độ cao.

5.1.2 Đánh giá so với mục tiêu

So sánh với các mục tiêu cụ thể đề ra ở Chương 1, hệ thống đã đáp ứng đầy đủ tất cả các mục tiêu trong phạm vi prototype. Đặc biệt, sự kết hợp giữa **dự báo chuỗi thời gian** và **nội suy không gian** trong cùng một hệ thống là điểm kỹ thuật nổi bật, chưa được tích hợp trong các giải pháp DIY tương tự hiện có.

5.1.3 Hạn chế

Tuy nhiên, đồ án vẫn còn một số hạn chế cần nhìn nhận thẳng thắn:

- **Cảm biến hạn chế:** Hệ thống hiện đo nhiệt độ và độ ẩm thông qua cảm biến cơ bản, chưa tích hợp các cảm biến đo chất lượng không khí chuyên sâu khác (như PM2.5, CO₂) do chi phí phần cứng thử nghiệm ban đầu.
- **Độ chính xác cảm biến:** Cảm biến thử nghiệm có sai số nhất định, chưa đạt chuẩn đo lường khí tượng hoặc y tế chính thức.
- **Dữ liệu huấn luyện hạn chế:** Mô hình dự báo được huấn luyện trên tập dữ liệu thu thập trong môi trường thực nghiệm quy mô nhỏ, cần được bổ sung nhiều dữ liệu thực tế đa dạng hơn để tăng độ tổng quát hóa.
- **Nội suy không gian với ít node:** Thuật toán nội suy đạt hiệu quả mượt mà và chính xác nhất khi có nhiều điểm đo. Với số lượng node thực nghiệm còn ít, kết quả bản đồ nhiệt phụ thuộc nhiều vào vị trí đặt node.
- **Chưa có đánh giá lâm sàng:** Các kết quả thực nghiệm chỉ mang tính tham khảo, chưa được kiểm chứng so sánh với các thiết bị tham chiếu tiêu chuẩn quốc gia.
- **Bảo mật kênh truyền:** Hệ thống thử nghiệm chưa mã hóa kênh truyền MQTT (chưa dùng MQTT over TLS), tiềm ẩn rủi ro bảo mật thông tin khi truyền trên mạng internet công cộng.
- **Bảo mật biến môi trường:** Trong giai đoạn phát triển ban đầu, thông tin xác thực cơ sở dữ liệu được hardcode trong code Python, tuy nhiên sau đó đã được khắc phục bằng cách cấu hình đọc tập trung từ tệp biến môi trường `.env`.

5.2 Hướng phát triển

Dựa trên những kết quả đạt được và hạn chế đã nhận diện, đồ án đề xuất các hướng phát triển cụ thể như sau:

5.2.1 Mở rộng khả năng cảm biến

Ưu tiên hàng đầu là tích hợp thêm các cảm biến chất lượng không khí thực tế như PMS5003 đo PM2.5/PM10, MH-Z19 đo CO₂ và cảm biến VOC. Đồng thời, nâng cấp cảm biến đo nhiệt độ/độ ẩm lên dòng có độ chính xác cao hơn để đảm bảo chất lượng dữ liệu đo đạc.

5.2.2 Nâng cao mô hình Machine Learning

Nghiên cứu áp dụng các kiến trúc mô hình học sâu chuỗi thời gian tiên tiến hơn (như Temporal Fusion Transformer hoặc N-BEATS) và xây dựng pipeline tự động tái huấn luyện (auto-retraining) khi phát hiện độ lệch mô hình (model drift) theo thời gian.

5.2.3 Cải thiện nội suy không gian

Nghiên cứu các phương pháp nội suy không gian kết hợp nhiều biến tương quan (Co-Kriging) hoặc tích hợp mô hình CFD (Computational Fluid Dynamics) cho các không gian phức tạp có nhiều vật cản hoặc cửa gió.

5.2.4 Tối ưu bảo mật và hạ tầng

Triển khai giao thức MQTT over TLS để mã hóa đường truyền dữ liệu từ node cảm biến lên máy chủ. Đóng gói toàn bộ hệ thống bằng Docker Compose để dễ dàng triển khai và kiểm thử tải trên quy mô hàng trăm node hoạt động đồng thời.

5.2.5 Tích hợp hệ sinh thái và mở rộng ứng dụng

Phát triển ứng dụng di động để người dùng nhận cảnh báo tức thời trên điện thoại, đồng thời thiết lập các kịch bản tự động kích hoạt điều hòa hoặc máy lọc không khí thông qua việc tích hợp với các hệ thống nhà thông minh phổ biến.

5.2.6 Thử nghiệm và đánh giá quy mô lớn

Tiến hành triển khai thực nghiệm với số lượng node nhiều hơn trong thời gian dài hơn, so sánh trực tiếp kết quả đo đạc với các trạm tham chiếu chuẩn để đánh giá khách quan sai số của hệ thống.

TÀI LIỆU THAM KHẢO

- [1] World Health Organization, “Who global air quality guidelines: Particulate matter (pm2.5 and pm10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide,” World Health Organization, Geneva, Tech. Rep., 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240034228>
- [2] Cục Kiểm soát ô nhiễm môi trường, Bộ Tài nguyên và Môi trường, “Báo cáo hiện trạng môi trường quốc gia 2023: Chuyên đề ô nhiễm không khí,” Bộ Tài nguyên và Môi trường Việt Nam, Tech. Rep., 2023.
- [3] J. Saini, M. Dutta, and G. Marques, “Indoor air quality monitoring systems based on internet of things: A systematic review,” *International Journal of Environmental Research and Public Health*, vol. 17, no. 14, p. 4942, 2020.
- [4] P. Kumar, L. Morawska, C. Martani, G. Biskos, M. Neophytou, S. Di Sabatino, M. Bell, L. Norford, and R. Britter, “The rise of low-cost sensing for managing air pollution in cities,” *Environment International*, vol. 75, pp. 199–205, 2015.
- [5] D. Margaritis, C. Keramydas, I. Papachristos, and D. Lambropoulou, “Calibration of low-cost gas sensors for air quality monitoring,” *Aerosol and Air Quality Research*, vol. 21, no. 11, 2021.
- [6] Xiaomi Inc., “Xiaomi mi air quality detector specifications,” <https://www.mi.com>, 2023, truy cập tháng 1/2026.
- [7] Q. Technology, “Qingping air monitor specifications,” <https://www.qingping.co/>, 2023, truy cập tháng 1/2026.
- [8] IQAir, “Iqair airvisual pro air quality monitor specifications,” <https://www.iqair.com>, 2024, truy cập tháng 1/2026.
- [9] *SenseAir S8 LP CO2 Sensor Module Datasheet*, Senseair AB, 2024, rev. 17, Article No. 004-0-0053. [Online]. Available: <https://www.senseair.com>
- [10] Bộ Tài nguyên và Môi trường Việt Nam, *Thông tư 10/2021/TT-BTNMT quy định kỹ thuật quan trắc môi trường và quản lý thông tin, dữ liệu quan trắc chất lượng môi trường*, 2021, ban hành ngày 30 tháng 06 năm 2021.
- [11] —, “Dự án đầu tư xây dựng mạng lưới quan trắc môi trường không khí (giai đoạn 1),” Báo cáo phê duyệt dự án, Tổng cục Môi trường, 2021, truy cập qua Trung tâm Quan trắc môi trường miền Bắc: <http://cem.gov.vn>.

- [12] S.-j. Zheng, C. Fu, B.-y. Wan, and Z.-g. Liu, “Comparative analysis of indoor humidity field simulation methods based on spatial interpolation,” *Water Saving Irrigation*, no. 2, pp. 7–10, 2019.
- [13] S. V. Belavadi, S. Rajagopal, and R. Ranjani, “Air quality forecasting using lstm rnn and wireless sensor networks,” *Procedia Computer Science*, vol. 170, pp. 241–248, 2020.
- [14] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [15] A. Banks and R. Gupta, “MQTT Version 3.1.1,” *OASIS Standard*, 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- [16] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [17] N. Cressie, “The origins of kriging,” *Mathematical Geology*, vol. 22, no. 3, pp. 239–252, 1990.
- [18] D. Shepard, “A two-dimensional interpolation function for irregularly-spaced data,” pp. 517–524, 1968.
- [19] Espressif Systems, “ESP32 technical reference manual,” https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf, 2024, truy cập tháng 1/2026.
- [20] Aosong Electronics Co., Ltd., “DHT11 humidity & temperature sensor datasheet,” <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>, 2023, truy cập tháng 1/2026.
- [21] PostgreSQL Global Development Group, “PostgreSQL documentation,” <https://www.postgresql.org/docs>, 2024, truy cập tháng 1/2026.
- [22] OpenJS Foundation, “Node.js,” <https://nodejs.org>, 2024, truy cập tháng 1/2026.
- [23] Socket.IO contributors, “Socket.IO,” <https://socket.io>, 2024, truy cập tháng 1/2026.
- [24] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com>, 2024, truy cập tháng 1/2026.
- [25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, 2019, vol. 32. [Online]. Available: <https://arxiv.org/abs/1912.01703>

- [26] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [27] Meta Platforms, Inc., “React,” <https://react.dev>, 2024, truy cập tháng 1/2026.
- [28] V. Agafonkin, “Leaflet — a javascript library for interactive maps,” <https://leafletjs.com>, 2024, truy cập tháng 1/2026.

PHỤ LỤC A.

PHỤ LỤC: MÃ NGUỒN QUAN TRỌNG

Phần phụ lục này lưu trữ mã nguồn chi tiết của các thành phần cốt lõi trong hệ thống AIoT giám sát và dự báo chất lượng không khí, phục vụ mục đích tham khảo kỹ thuật.

A.1 Mã nguồn Firmware ESP32

```
1 // sensor/sensor.ino ( L c t r c h p h n v n g l p c h n h )
2 #include <WiFi.h>
3 #include <PubSubClient.h>
4 #include <ArduinoJson.h>
5 #include <DHT.h>
6 #include <Preferences.h>
7 #include <WiFiManager.h>
8
9 #define DHTPIN 4
10 #define DHTTYPE DHT11
11 #define SEND_INTERVAL 5000 // 5 gi y
12
13 DHT dht(DHTPIN, DHTTYPE);
14 WiFiClient wifiClient;
15 PubSubClient mqttClient(wifiClient);
16
17 String deviceApiKey = "";
18 int deviceId = -1;
19 bool provisioned = false;
20
21 void setup() {
22     dht.begin();
23     loadCredentials(); // c device_id, api_key t Flash
24     WiFiManager wm;
25     wm.autoConnect("Sensor-Setup-[MAC]");
26     mqttClient.setServer(MQTT_BROKER, 1883);
27     mqttClient.setCallback(mqttCallback);
28 }
29
30 void loop() {
```

```

31  if (!provisioned) { sendProvisionRequest(); return; }
32  if (!mqttClient.connected()) reconnectWithAuth();
33  mqttClient.loop();
34
35  // G i   d   l i u   t h e o   c h u   k   S E N D _ I N T E R V A L
36  if (millis() - lastSend >= SEND_INTERVAL) {
37      float temp = dht.readTemperature();
38      float hum  = dht.readHumidity();
39      float aqi  = generateMockAqi(); // G i   l   p   A Q I
40
41      if (!isnan(temp) && !isnan(hum)) {
42          publishData(temp, hum, aqi);
43      }
44      lastSend = millis();
45  }
46 }
47
48 void publishData(float temp, float hum, float aqi) {
49     StaticJsonDocument<128> doc;
50     doc["temperature"] = temp;
51     doc["humidity"]    = hum;
52     doc["aqi"]         = aqi;
53     char payload[128];
54     serializeJson(doc, payload);
55
56     String topic = "device/" + deviceApiKey + "/data";
57     mqttClient.publish(topic.c_str(), payload);
58 }

```

Listing A.1: Luồng xử lý chính của Firmware ESP32

A.2 Cấu trúc Cơ sở dữ liệu và Dịch vụ Cảnh báo Backend

```

1  -- B n g   n g   i   d   n g
2  CREATE TABLE users (
3      id          SERIAL PRIMARY KEY,
4      name        VARCHAR(100),
5      email       VARCHAR(100) UNIQUE NOT NULL,
6      password    VARCHAR(255) NOT NULL,      -- bcrypt hash
7      role        VARCHAR(20) DEFAULT 'user',

```

```

8      created_at TIMESTAMP DEFAULT NOW()
9  );
10
11  -- B ng ph ng/khu v c
12  CREATE TABLE rooms (
13      id          SERIAL PRIMARY KEY,
14      name        VARCHAR(100),
15      description  TEXT,
16      center_lat   REAL,
17      center_lng   REAL,
18      width_m      REAL,
19      length_m     REAL,
20      owner_user_id INTEGER REFERENCES users(id),
21      created_at   TIMESTAMP DEFAULT NOW()
22  );
23
24  -- B ng thit b IoT
25  CREATE TABLE devices (
26      id          SERIAL PRIMARY KEY,
27      name        VARCHAR(100),
28      description  TEXT,
29      owner_user_id INTEGER REFERENCES users(id),
30      api_key      VARCHAR(64) UNIQUE NOT NULL,
31      notify_email  BOOLEAN DEFAULT TRUE,
32      temp_high     REAL DEFAULT 35.0,
33      temp_low      REAL DEFAULT 10.0,
34      hum_high      REAL DEFAULT 80.0,
35      hum_low       REAL DEFAULT 20.0,
36      aqi_high      REAL,
37      aqi_low       REAL,
38      room_id       INTEGER REFERENCES rooms(id),
39      mac_address   VARCHAR(17) UNIQUE,
40      x             DOUBLE PRECISION,
41      y             DOUBLE PRECISION,
42      lat           DOUBLE PRECISION,
43      lng           DOUBLE PRECISION,
44      firmware_version VARCHAR(50),
45      chip_model    VARCHAR(50),
46      created_at    TIMESTAMP DEFAULT NOW()
47  );
48

```

```

49 -- B n g   d   l i u   c m   b i n
50 CREATE TABLE readings (
51     id          SERIAL PRIMARY KEY,
52     device_id   INTEGER REFERENCES devices(id),
53     temperature REAL NOT NULL,
54     humidity    REAL NOT NULL,
55     aqi         REAL NOT NULL DEFAULT 0,
56     timestamp   TIMESTAMP DEFAULT NOW()
57 );
58
59 -- Index t n g   t c   truy   v n   ch u i   t h i   gian
60 CREATE INDEX idx_readings_device_time
61     ON readings(device_id, timestamp DESC);
62
63 -- B n g   c n h   b o
64 CREATE TABLE alerts (
65     id          SERIAL PRIMARY KEY,
66     device_id   INTEGER REFERENCES devices(id),
67     type        VARCHAR(50),
68     value       REAL,
69     message     TEXT,
70     acknowledged_by INTEGER REFERENCES users(id),
71     acknowledged_at TIMESTAMP,
72     timestamp   TIMESTAMP DEFAULT NOW()
73 );

```

Listing A.2: Schema cơ sở dữ liệu PostgreSQL

```

1 // server/services/alertService.js
2 async function check(device, reading) {
3     const { temperature, humidity } = reading;
4     const alerts = [];
5
6     if (temperature > device.temp_high)
7         alerts.push({ type: 'temp_high', value: temperature,
8             message: `N h i t           ${temperature}C v   t
9                 n g   n g   ${device.temp_high}C` });
10
11     if (temperature < device.temp_low)
12         alerts.push({ type: 'temp_low', value: temperature,
13             message: `N h i t           ${temperature}C d   i
14                 n g   n g   ${device.temp_low}C` });

```

```

13
14     if (humidity > device.hum_high)
15         alerts.push({ type: 'hum_high', value: humidity,
16             message: `          m  ${humidity}% v    t    n g    ng  ${
17                 device.hum_high}%` });
18
19     if (humidity < device.hum_low)
20         alerts.push({ type: 'hum_low', value: humidity,
21             message: `          m  ${humidity}% d    i    n g    ng  ${
22                 device.hum_low}%` });
23
24     for (const alert of alerts) {
25         // L u v o DB
26         await db.query(
27             `INSERT INTO alerts
28             (device_id, type, value, message, timestamp)
29             VALUES ($1,$2,$3,$4,NOW())`,
30             [device.id, alert.type,
31                 alert.value, alert.message]);
32
33         // G i email n u t h i t b    b t t h n g b o
34         if (device.notify_email)
35             await emailService.sendAlert(device, alert);
36     }
37 }

```

Listing A.3: Logic dịch vụ kiểm tra ngưỡng cảnh báo (Backend Node.js)

A.3 Mã nguồn Python Machine Learning Service

```

1 # lstm_train/train.py (Tr ch    on    quy tr nh)
2 import torch
3 import torch.nn as nn
4 from sklearn.preprocessing import MinMaxScaler
5 import joblib
6
7 # 1. Load data from PostgreSQL
8 data = query("SELECT temperature, humidity, aqi FROM readings ORDER
9             BY timestamp")

```

```

10 # 2. Min-Max normalization, save scaler for inference reuse
11 scaler = MinMaxScaler()
12 data_scaled = scaler.fit_transform(data)
13 joblib.dump(scaler, 'scaler.pkl')
14
15 # 3. Create (X, y) pairs using sliding window
16 X, y = create_sequences(data_scaled, window=30, horizon=15)
17
18 # 4. Define LSTM architecture (PyTorch)
19 class LSTMModel(nn.Module):
20     def __init__(self):
21         super().__init__()
22         self.lstm = nn.LSTM(input_size=3,
23                             hidden_size=64,
24                             num_layers=2,
25                             batch_first=True,
26                             dropout=0.3) # c p n h t
27                                     dropout=0.3
28         self.fc = nn.Linear(64, 15 * 3)
29
30     def forward(self, x):
31         out, _ = self.lstm(x)
32         return self.fc(out[:, -1, :]).view(-1, 15, 3)
33
34 # 5. Train with Adam optimizer, MSE loss
35 model = LSTMModel()
36 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
37 criterion = nn.MSELoss()
38
39 for epoch in range(num_epochs):
40     pred = model(X_train)
41     loss = criterion(pred, y_train)
42     optimizer.zero_grad()
43     loss.backward()
44     optimizer.step()
45
46 # 6. Save model weights
47 torch.save(model.state_dict(), 'lstm_model.pth')

```

Listing A.4: Quy trình huấn luyện mô hình dự báo LSTM

```

1 # forecast/time_forecast/main.py ( P h n API inference)

```

```

2 from fastapi import FastAPI, Depends
3 import torch
4 import joblib
5
6 model = LSTMModel()
7 model.load_state_dict(torch.load('lstm_model.pth'))
8 model.eval()
9 scaler = joblib.load('scaler.pkl')
10
11 @app.get("/forecast")
12 async def forecast(device_id: int, db=Depends(get_db)):
13     # 1. Fetch 30 latest records from PostgreSQL
14     rows = await db.fetch(
15         "SELECT temperature, humidity, aqi FROM readings WHERE
16             device_id=$1 ORDER BY timestamp DESC LIMIT 30",
17         device_id)
18
19     # 2. Normalize using saved scaler
20     data = scaler.transform(rows)
21
22     # 3. Create input tensor
23     X = torch.tensor(data).unsqueeze(0).float()
24
25     # 4. Inference (no_grad to save memory)
26     with torch.no_grad():
27         pred = model(X)
28
29     # 5. Inverse transform to real values
30     result = scaler.inverse_transform(pred.squeeze(0).numpy())
31
32     return {"forecast": result.tolist()}

```

Listing A.5: Phục vụ suy luận mô hình LSTM qua FastAPI Service

```

1 # forecast/spatial_forecast.py
2 from pykrige.ok import OrdinaryKriging
3 import numpy as np
4
5 def spatial_forecast(nodes: list[dict], room: dict):
6     # 1. Create virtual nodes at 4 room corners via IDW
7     corners = get_room_corners(room)
8     virtual_nodes = [idw_estimate(c, nodes) for c in corners]

```

```

9      all_nodes = nodes + virtual_nodes
10
11      # 2. Extract coordinates and values
12      lats = [n["lat"] for n in all_nodes]
13      lngs = [n["lng"] for n in all_nodes]
14      temps = [n["temperature"] for n in all_nodes]
15      hums = [n["humidity"] for n in all_nodes]
16
17      # 3. Create 40x50 target grid
18      grid_lat = np.linspace(min(lats), max(lats), 40)
19      grid_lng = np.linspace(min(lngs), max(lngs), 50)
20
21      # 4. Kriging interpolation for temperature
22      ok_temp = OrdinaryKriging(
23          lngs, lats, temps,
24          variogram_model='gaussian',
25          verbose=False, enable_plotting=False)
26      z_temp, _ = ok_temp.execute('grid', grid_lng, grid_lat)
27
28      # 5. Kriging interpolation for humidity
29      ok_hum = OrdinaryKriging(
30          lngs, lats, hums,
31          variogram_model='gaussian',
32          verbose=False, enable_plotting=False)
33      z_hum, _ = ok_hum.execute('grid', grid_lng, grid_lat)
34
35      return {
36          "grid_lat": grid_lat.tolist(),
37          "grid_lng": grid_lng.tolist(),
38          "temperature_grid": z_temp.tolist(),
39          "humidity_grid": z_hum.tolist()
40      }

```

Listing A.6: Thuật toán nội suy không gian địa thống kê Kriging/IDW